

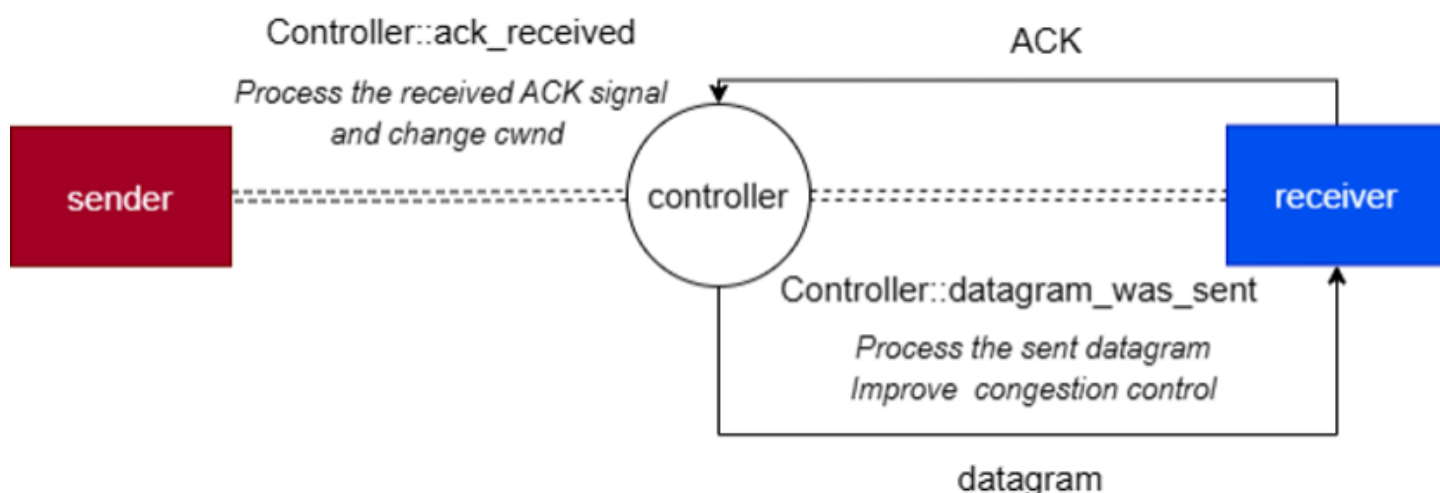
Transhub安装指南2026版

💡 PDF/Word版文档不方便复制多行命令，且图片较为模糊，可登录transhub查看[在线版本](#)（《[Transhub竞赛指南](#)》中"Transhub安装指南"部分），获取更好的阅读体验。

硬件环境：一台Linux主机（在本文中，Linux主机包括运行Linux系统的实体机、虚拟机或云服务器）

Linux版本要求：Ubuntu GNU/Linux 14.04以上

拥塞控制算法的运行示意图如下：



本教程提供两种安装方式，两种方式**任选其一**：

- 第一种方法为使用Linux虚拟机/主机，安装相关依赖，下载transhub代码并编译，编译完成后即可运行实验。
- 第二种方法为使用已配置好的Docker镜像，镜像里已装好相关依赖和代码，**开箱即用**，可**直接**运行实验。

❗ 由于以上部分依赖仅有x86架构的包，因此使用**Mac M芯片**的同学需要注意，如果使用第一种方法自行安装虚拟机，需安装x86架构的Linux系统，使用Arm架构的虚拟机将无法安装所有依赖。建议Mac M芯片同学使用**Docker方法**。

(一) 安装transhub

1. 方式一：使用Linux虚拟机/主机直接安装transhub

★Windows系统推荐使用此方式

1.1. 安装Linux虚拟机（若直接使用Linux物理机可跳过此步）

- 下载VMware软件

参考教程：

[VMware Workstation Pro 17官网下载安装教程_vmware17pro下载-CSDN博客](#)

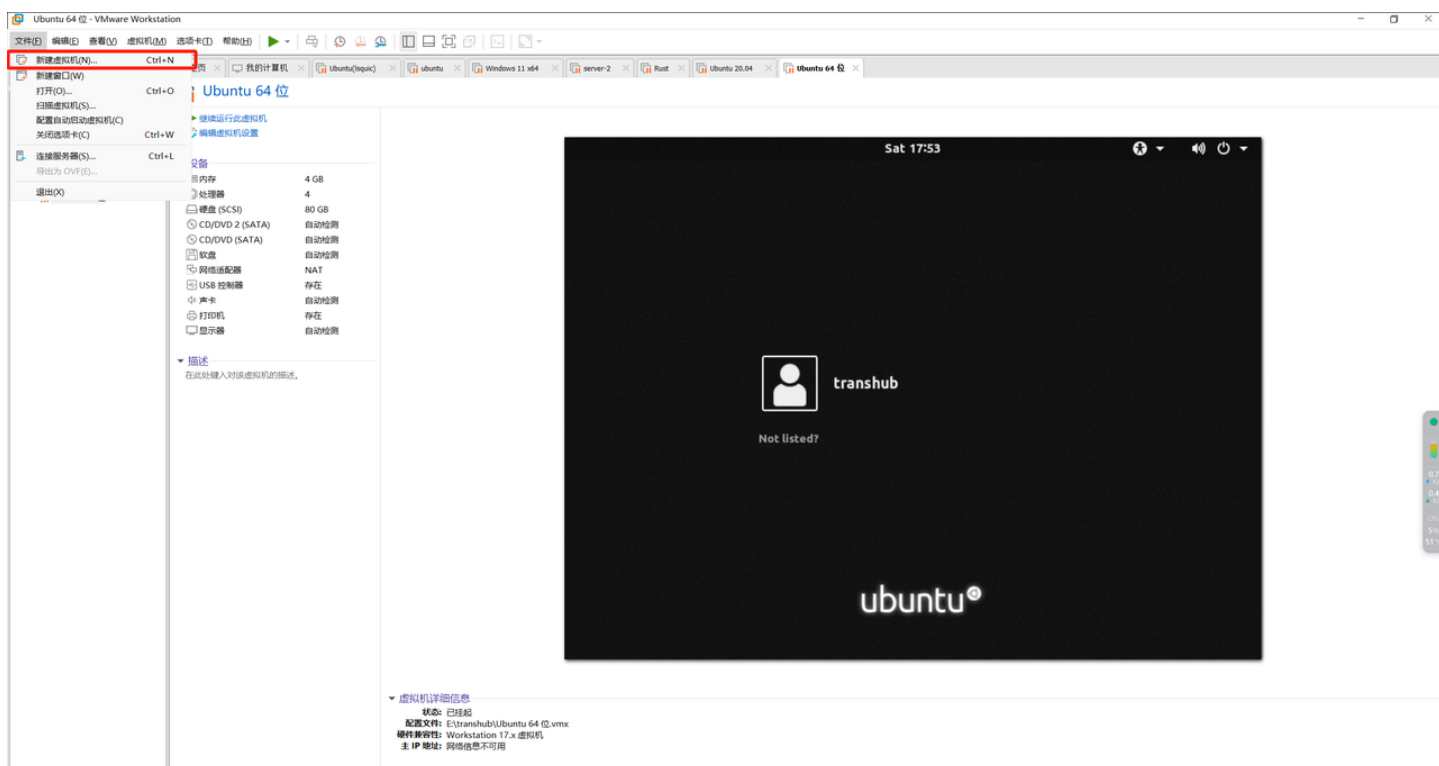
官方下载链接：

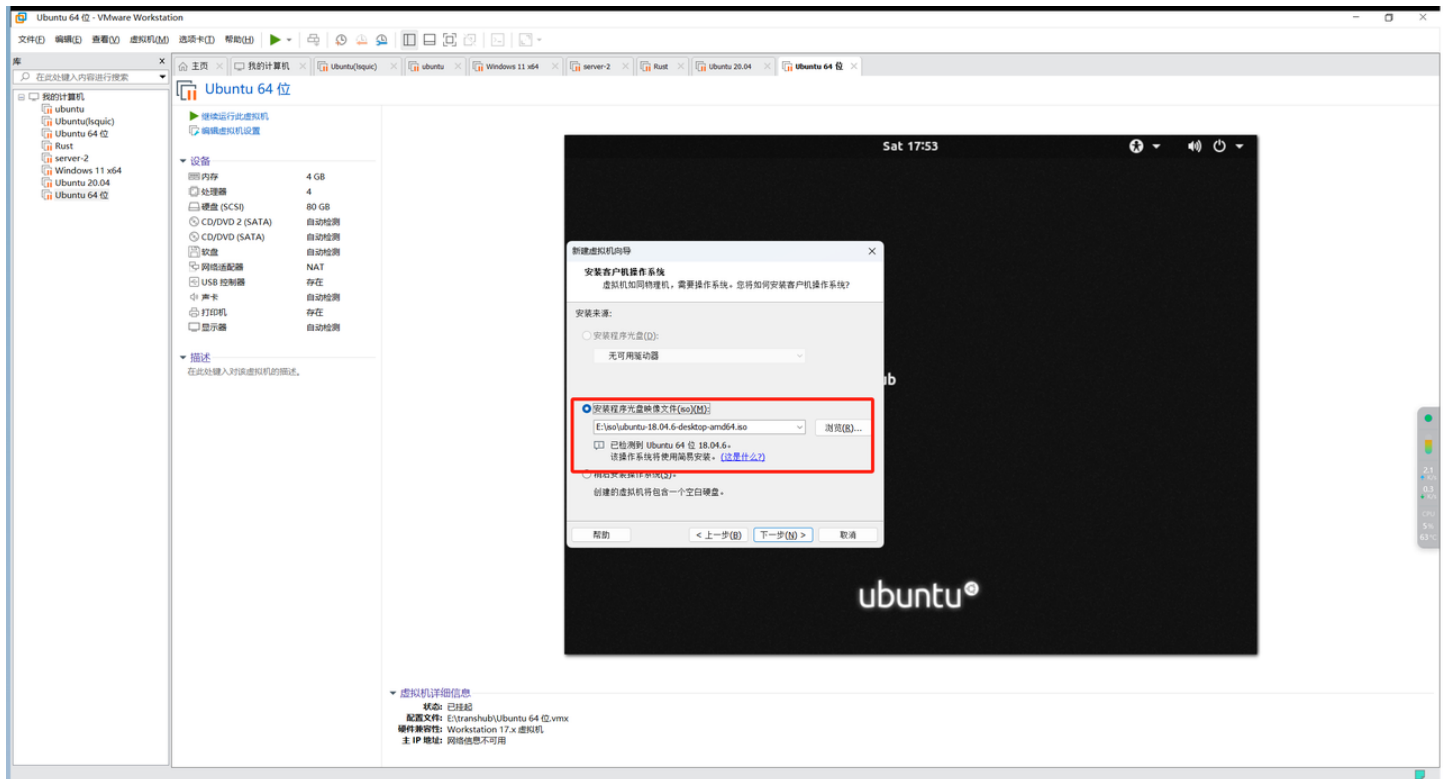
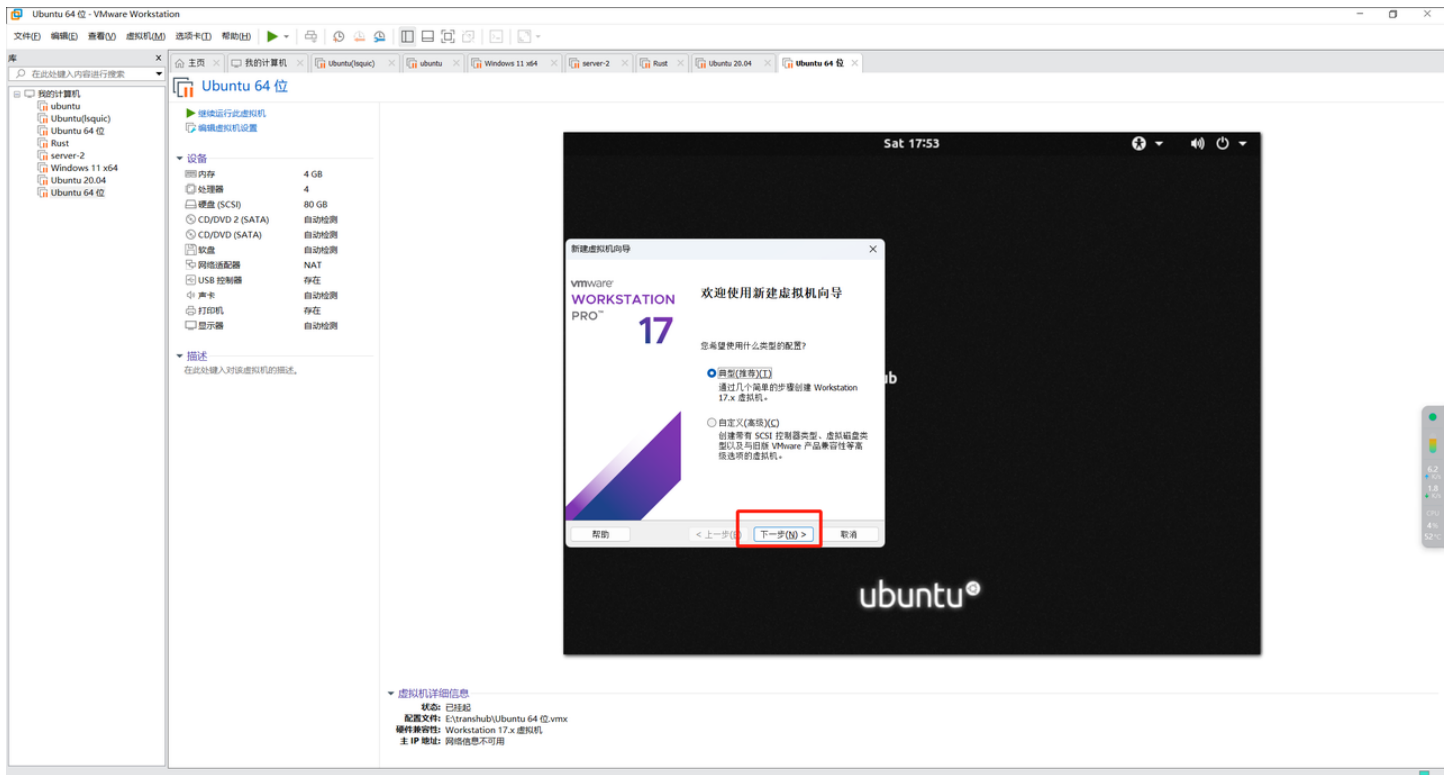
[https://support.broadcom.com/group/ecx/productdownloads?
subfamily=VMware%20Workstation%20Pro](https://support.broadcom.com/group/ecx/productdownloads?subfamily=VMware%20Workstation%20Pro)

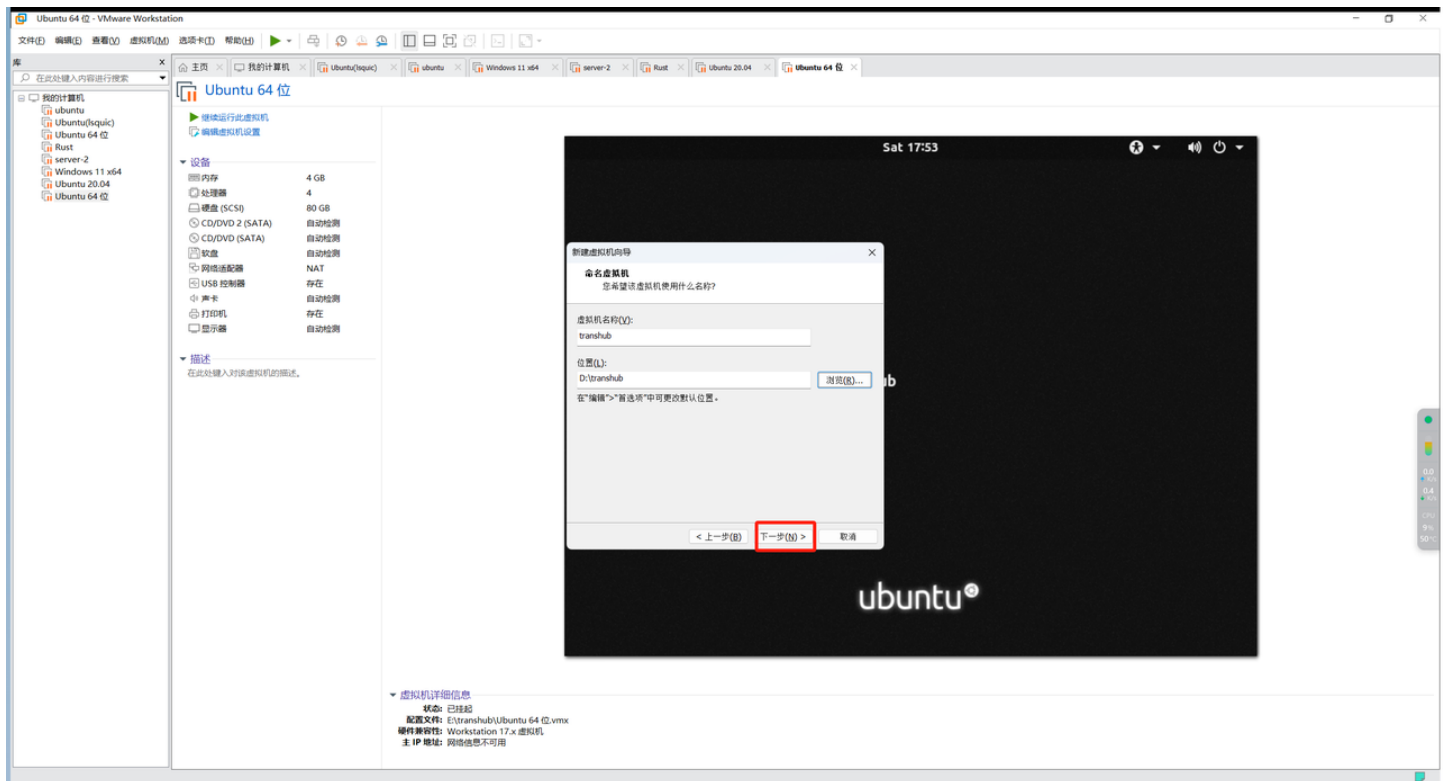
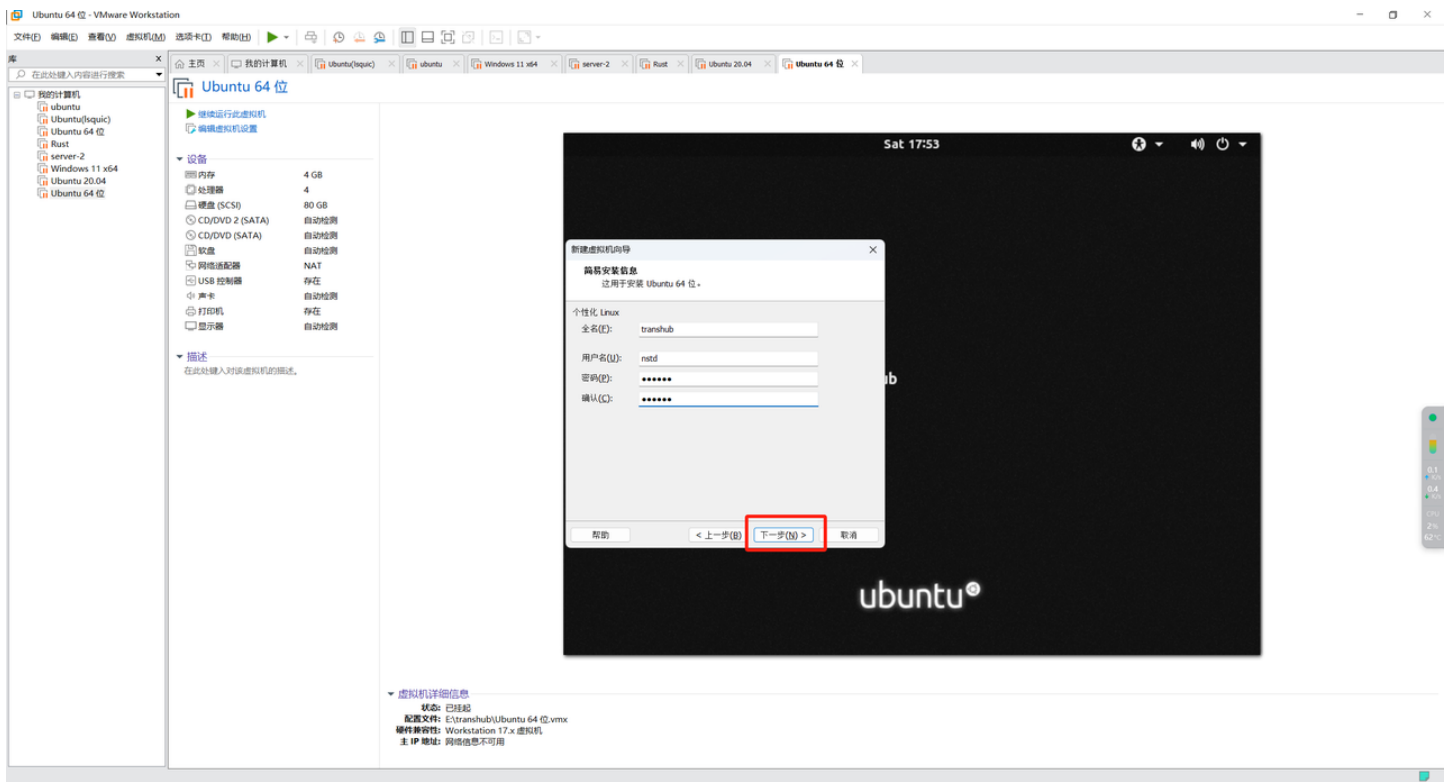
- 下载Ubuntu 18.04的镜像（Ubuntu20版本可能存在依赖无法安装的问题，建议使用**18**版本）

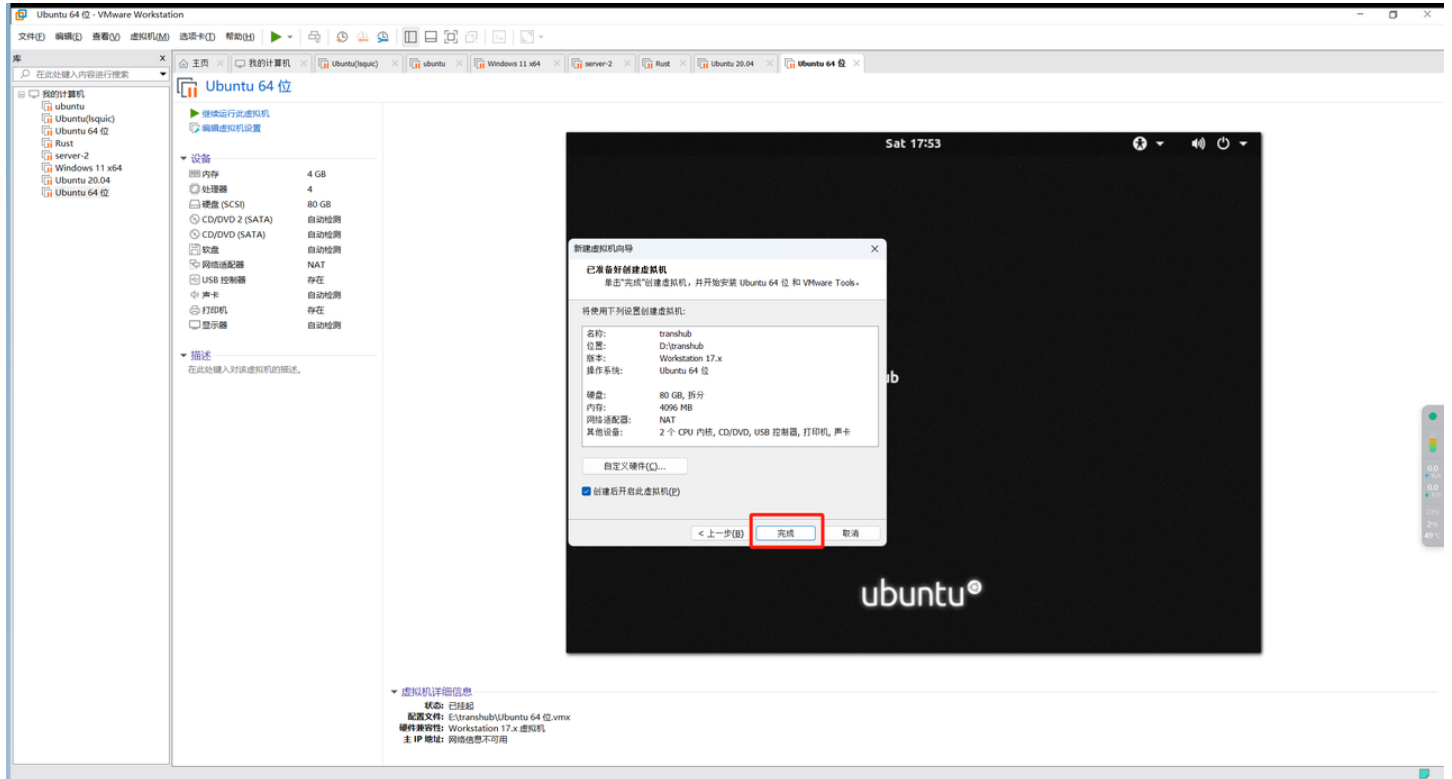
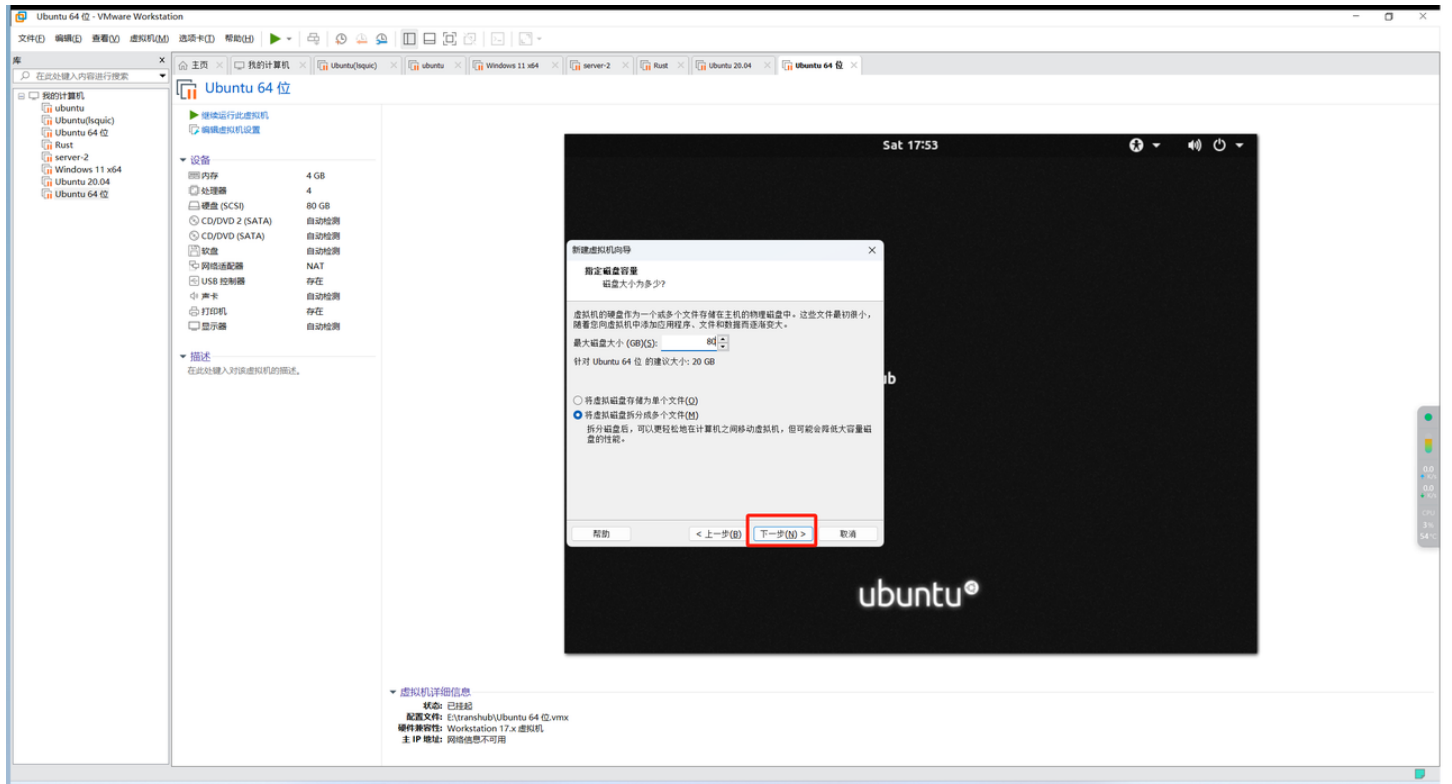
[Index of /ubuntu-releases/18.04/](#) | [清华大学开源软件镜像站](#) | [Tsinghua Open Source Mirror](#)

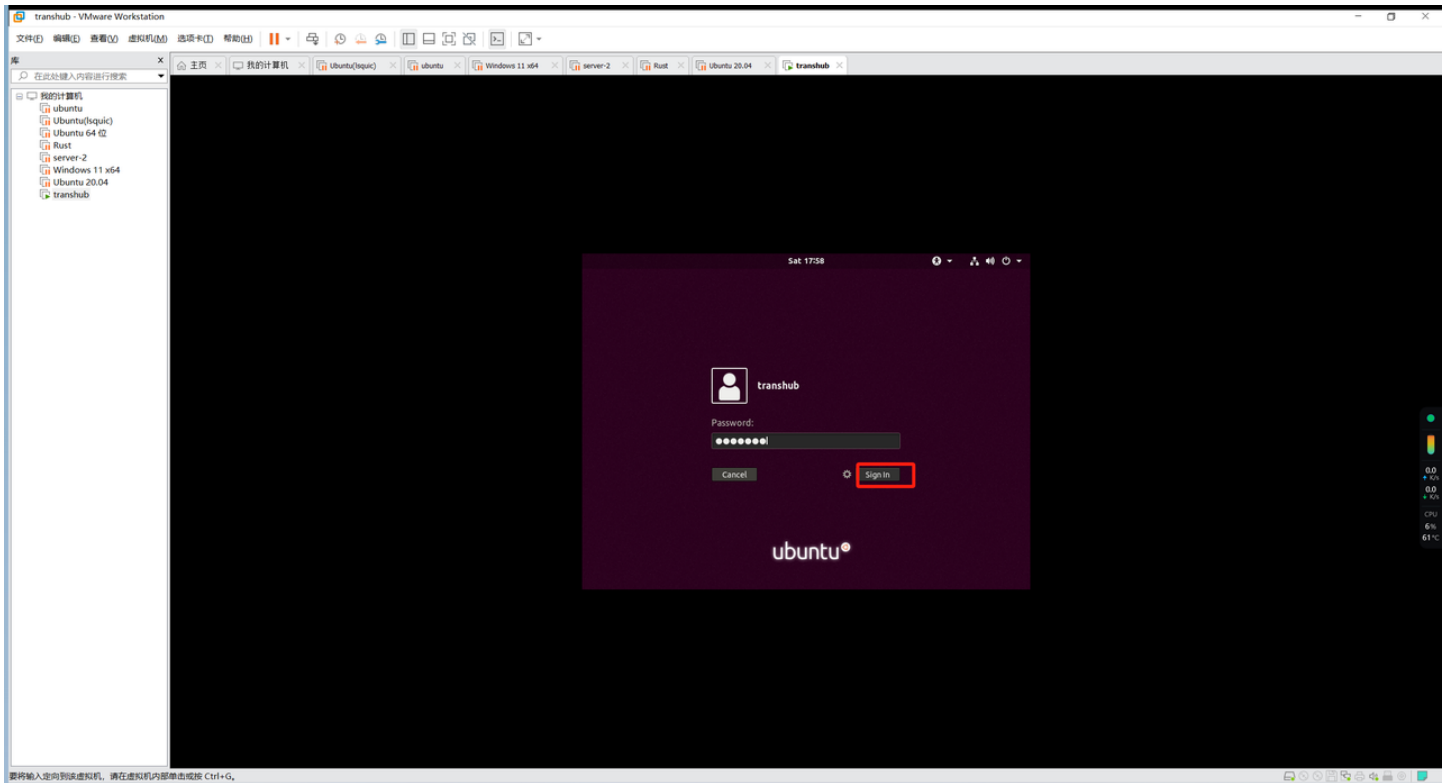
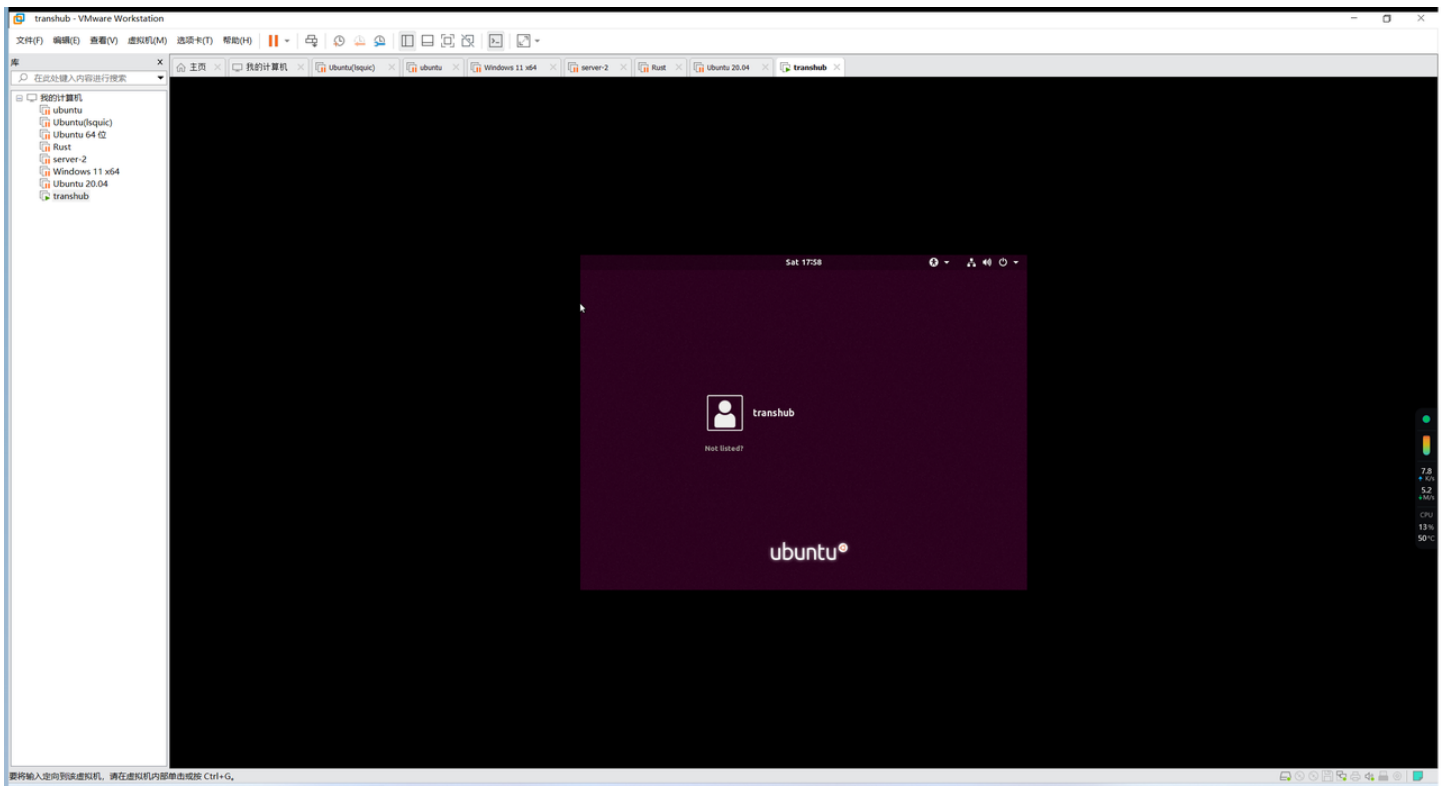
- 使用VMware创建Ubuntu 18.04的虚拟机



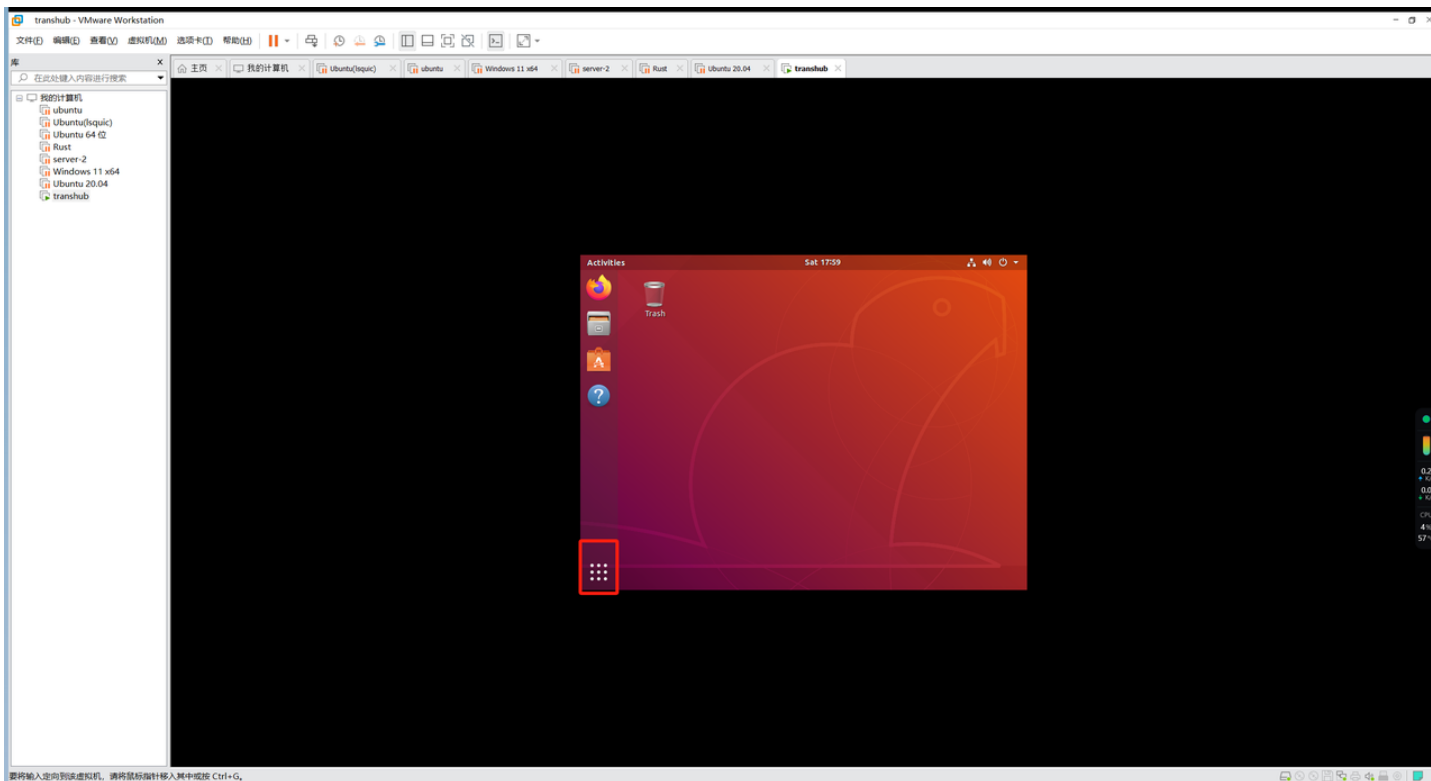




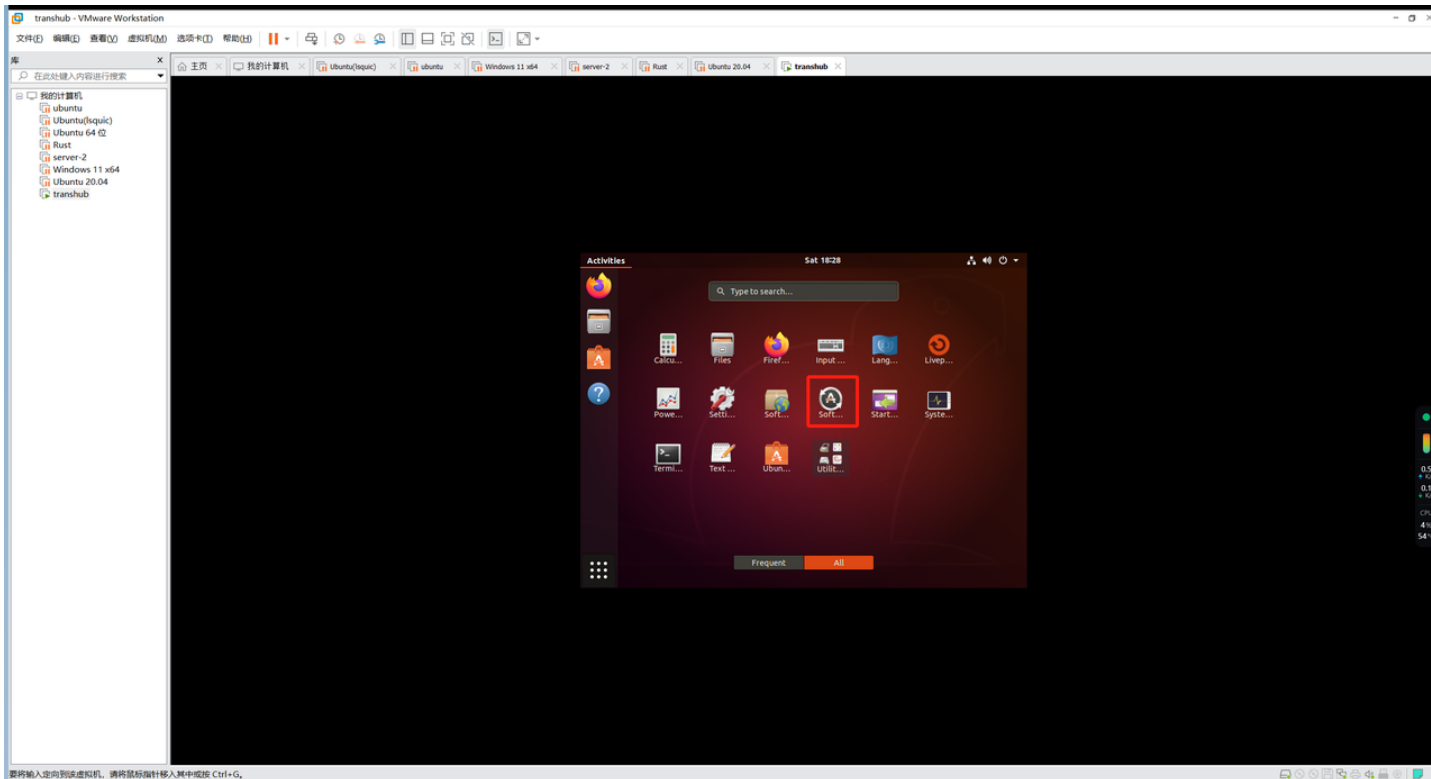




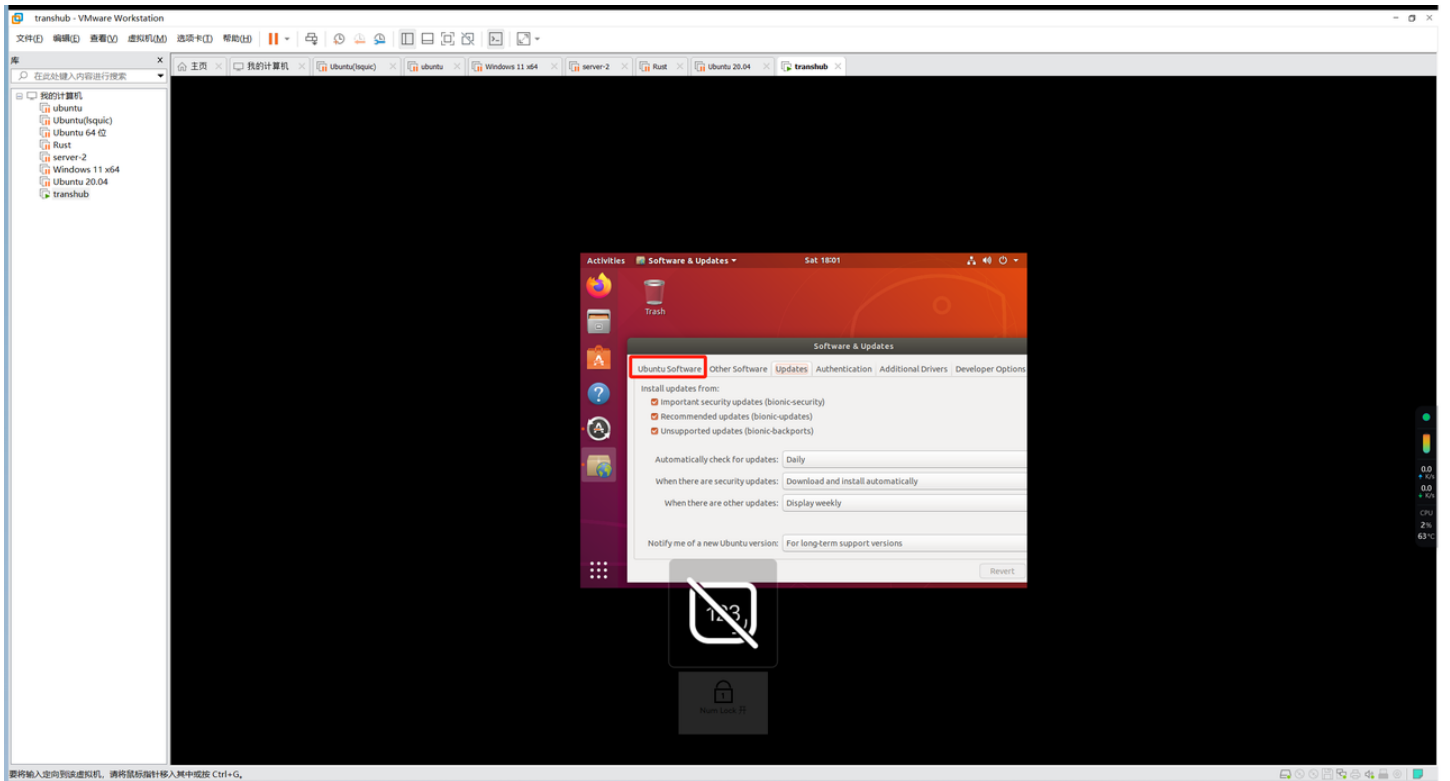
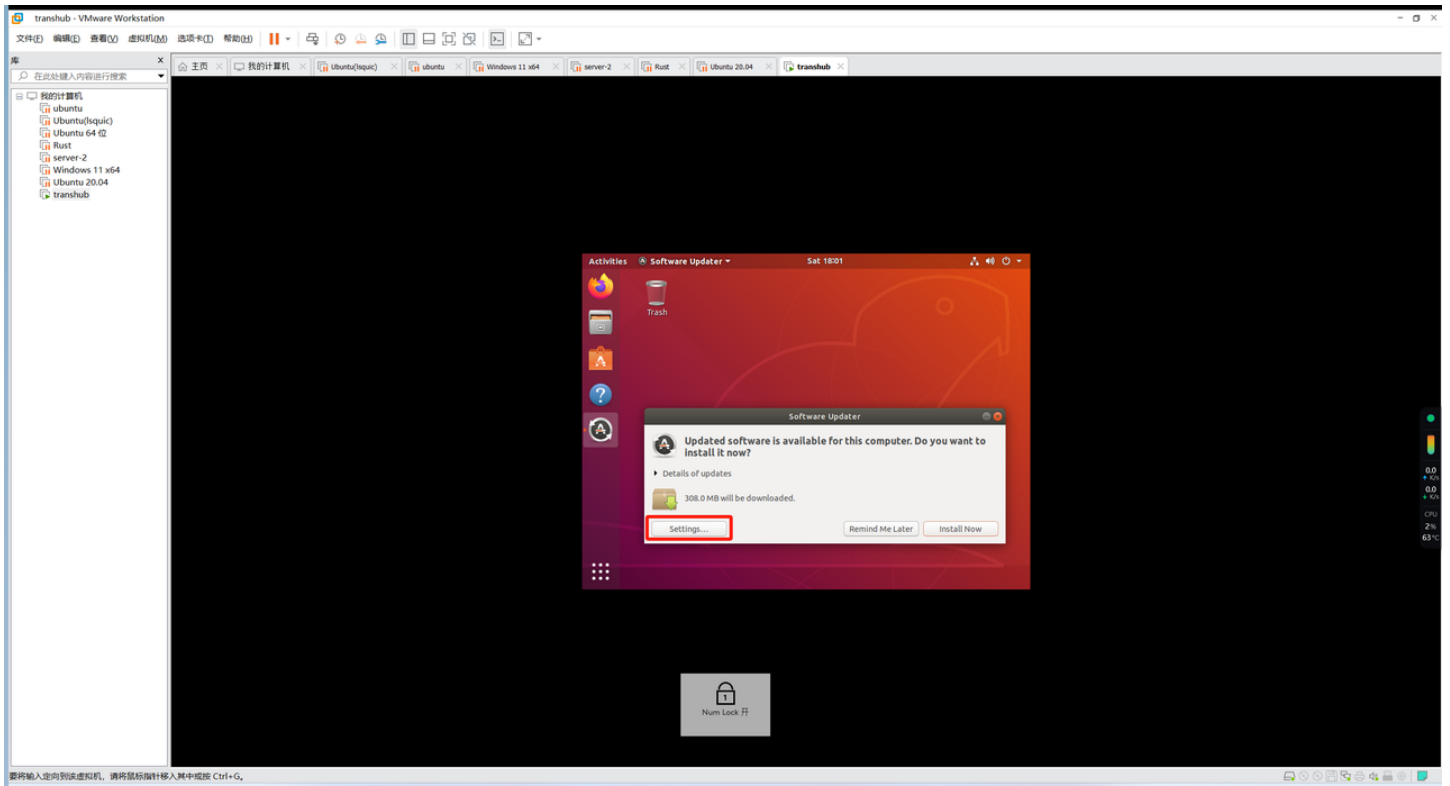
- 将依赖包下载源站换为国内网站，避免因为网络问题导致依赖包下载失败

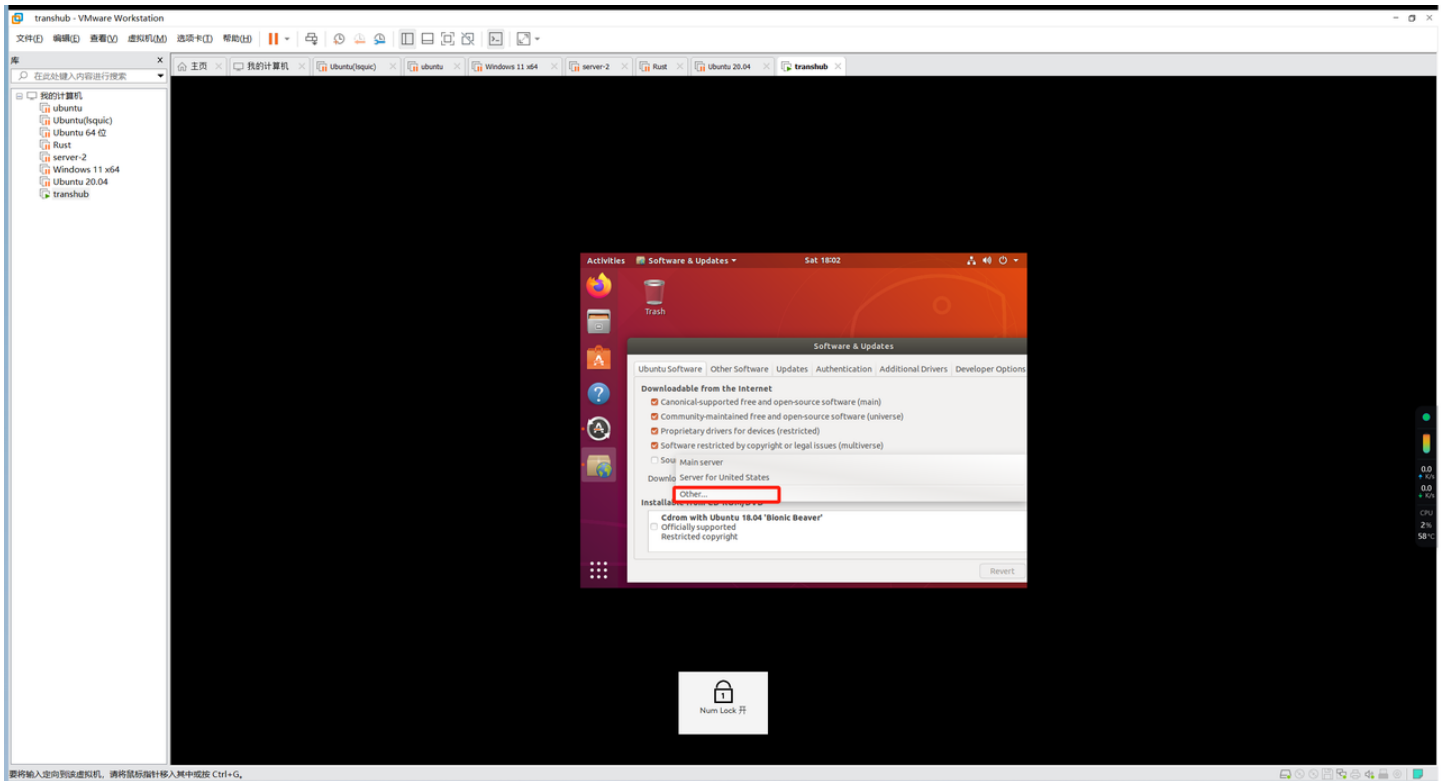
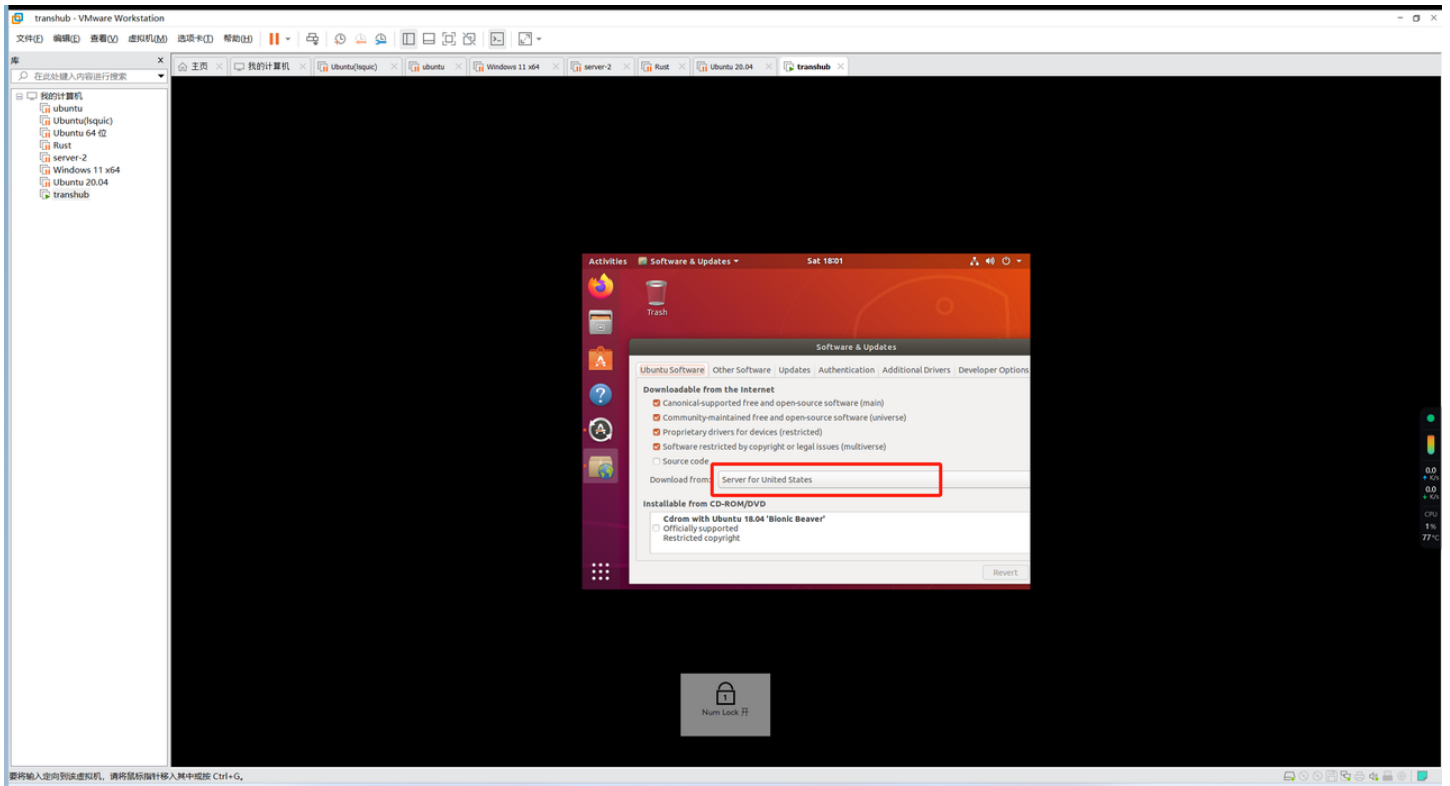


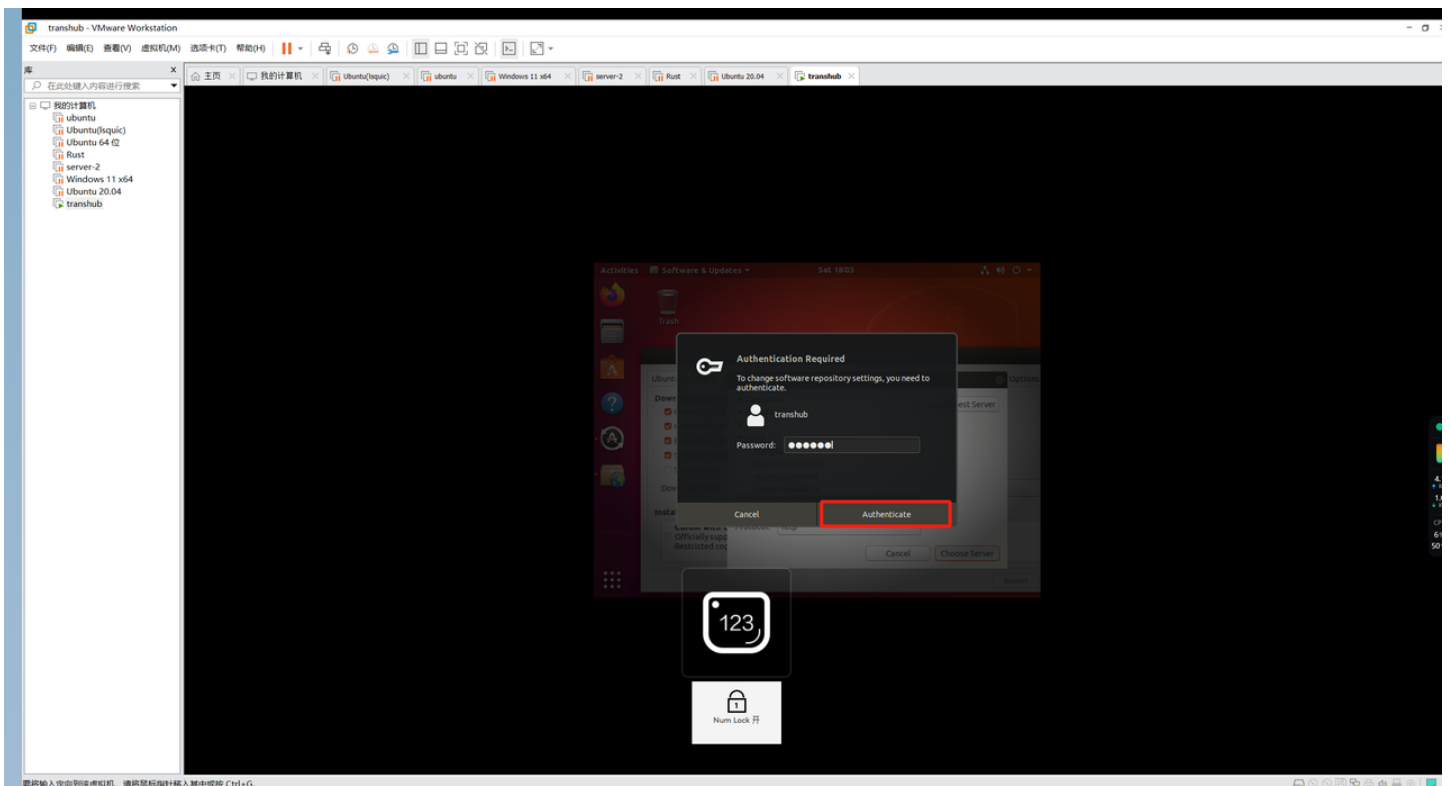
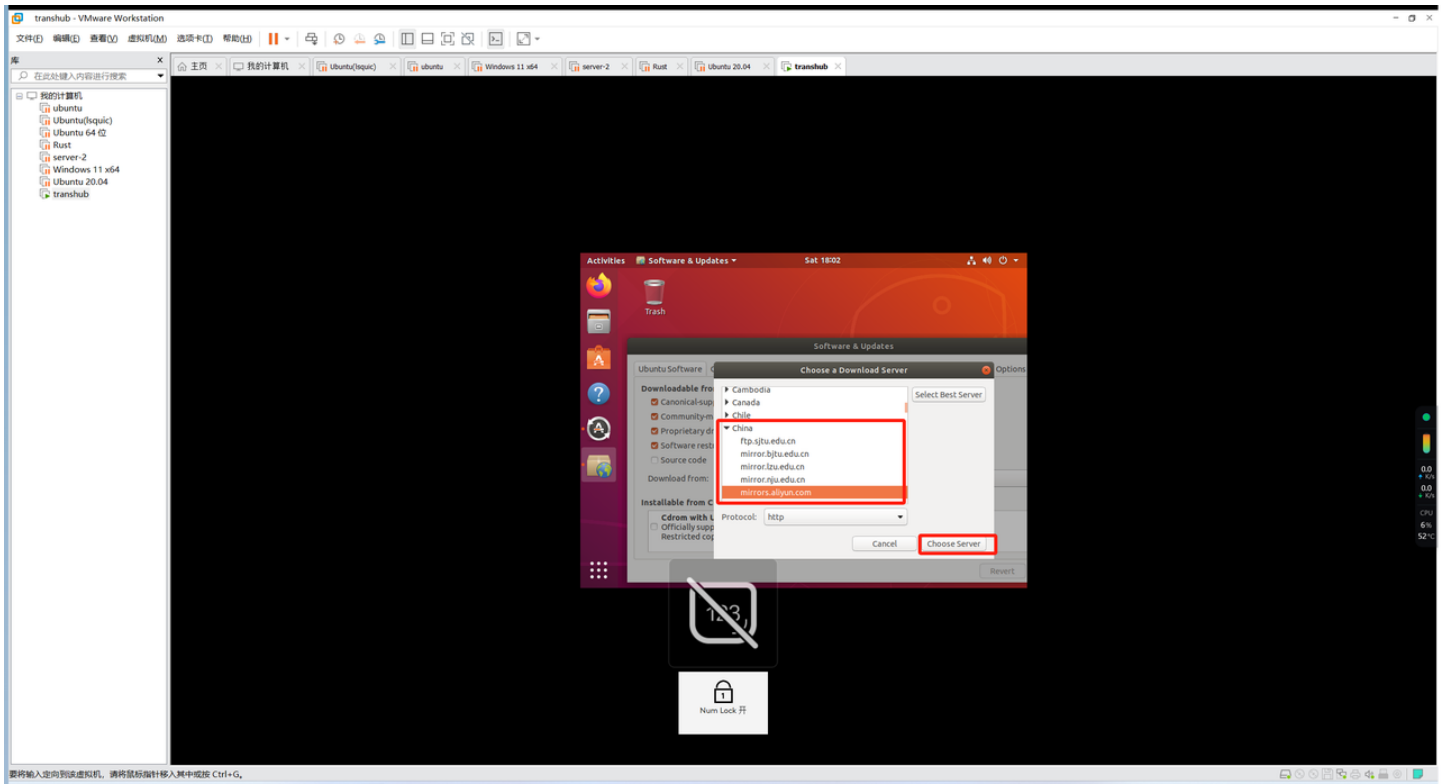
要将输入定向到该虚拟机，请将鼠标指针移入其中或按 Ctrl+G。

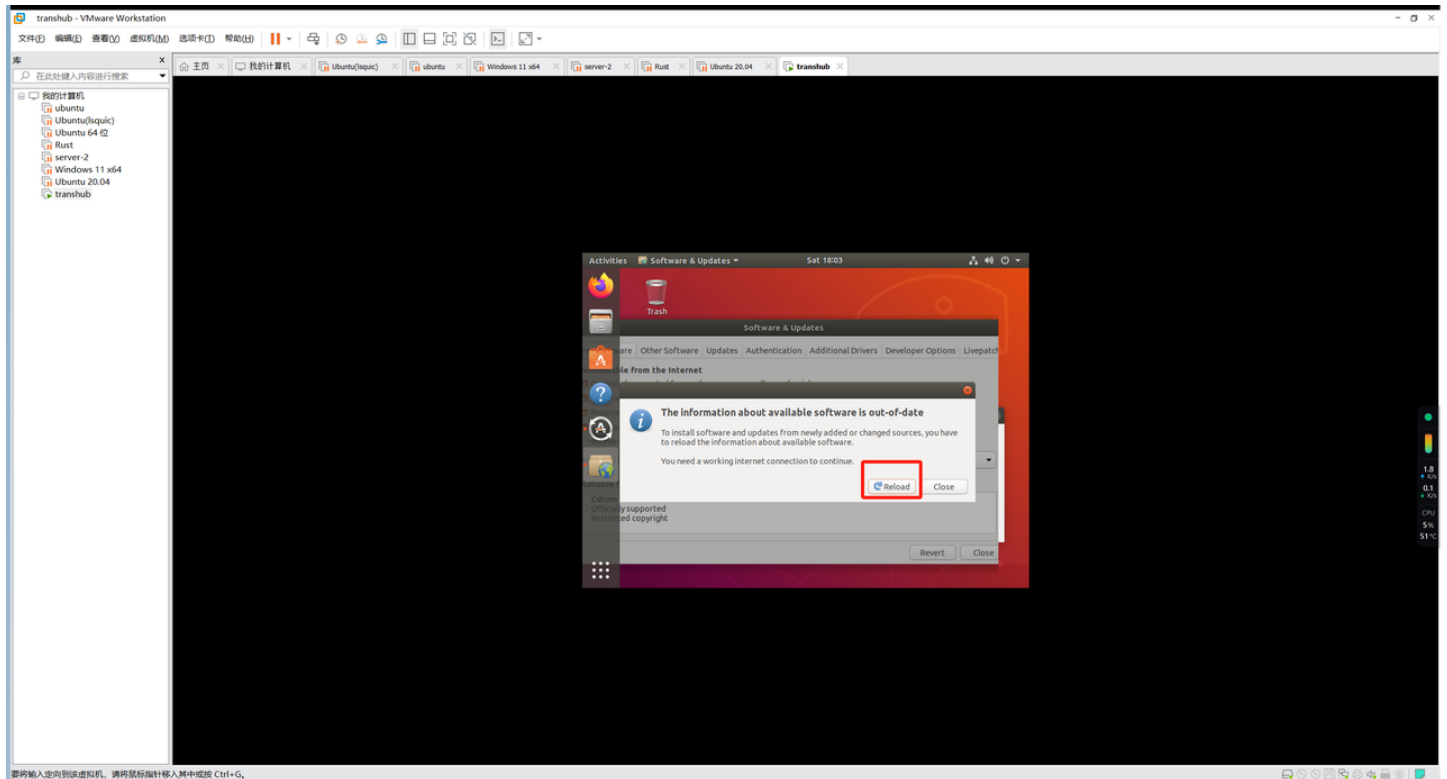
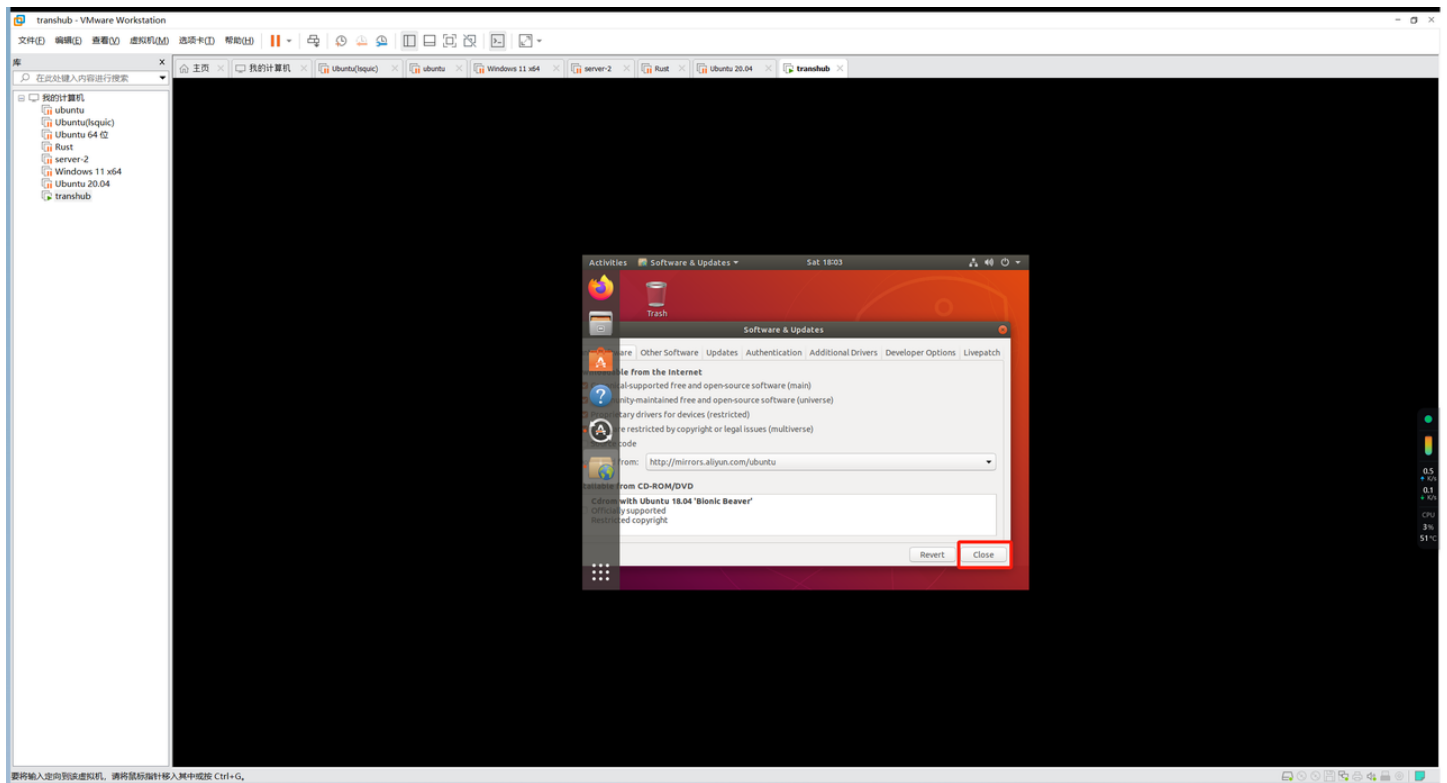


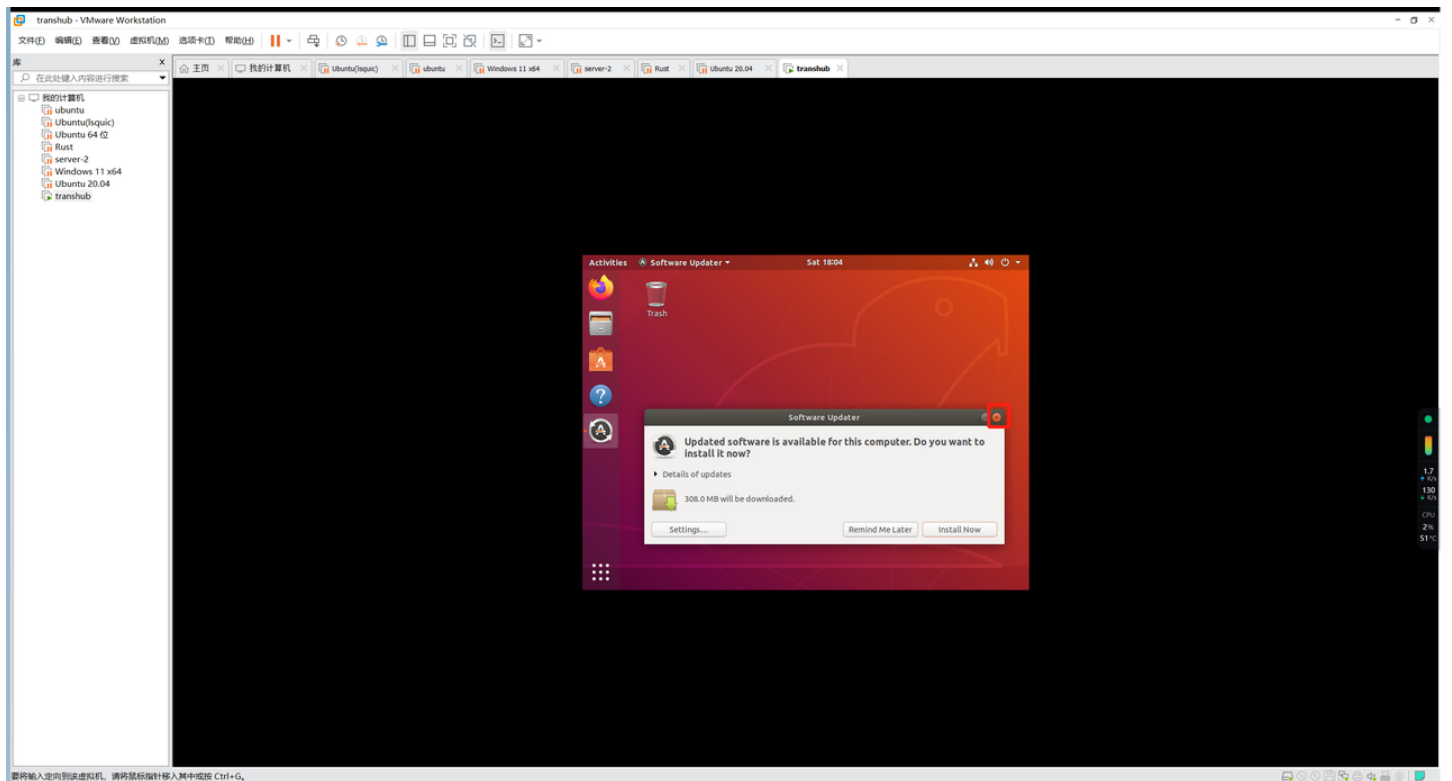
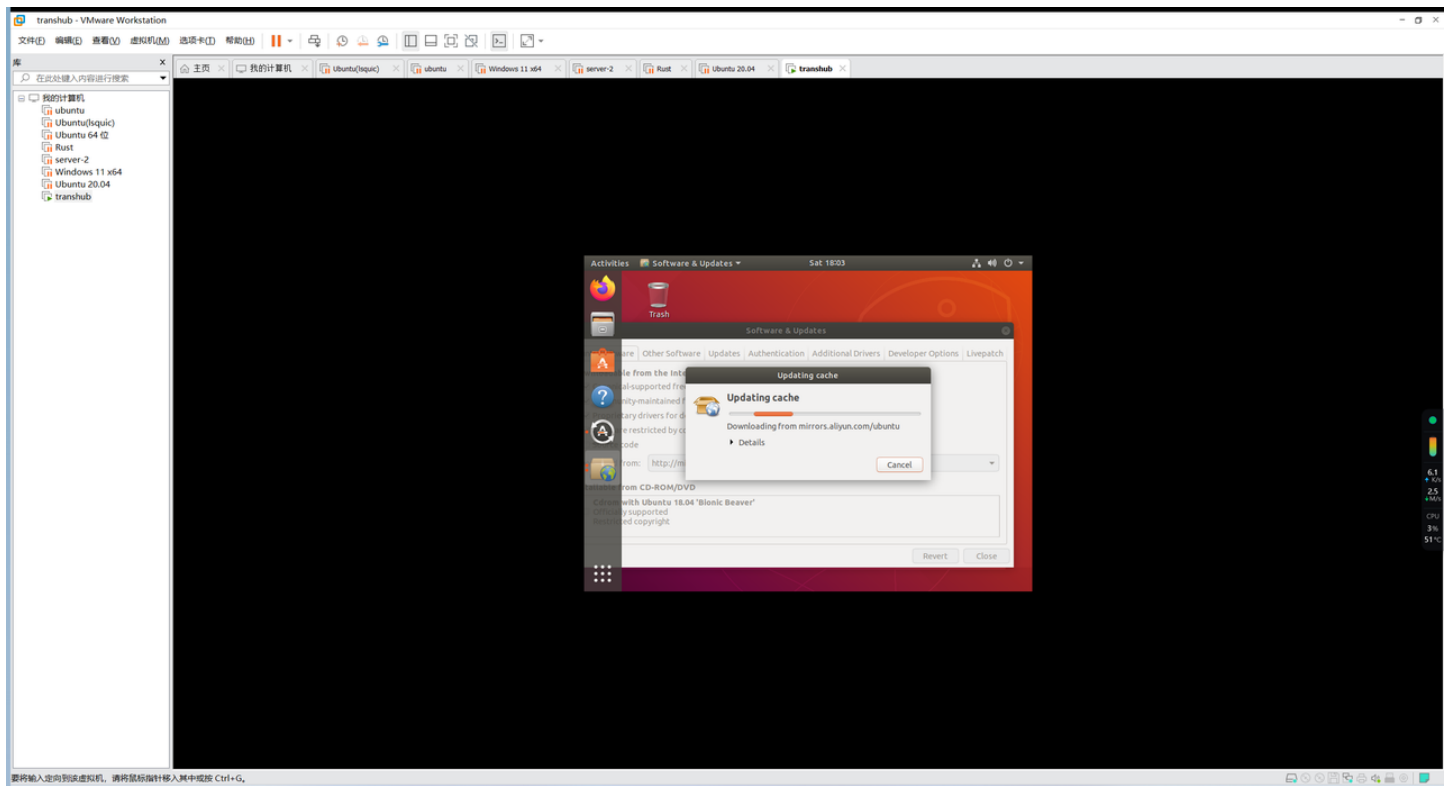
要将输入定向到该虚拟机，请将鼠标指针移入其中或按 Ctrl+G。











1.2. 安装Mahimahi

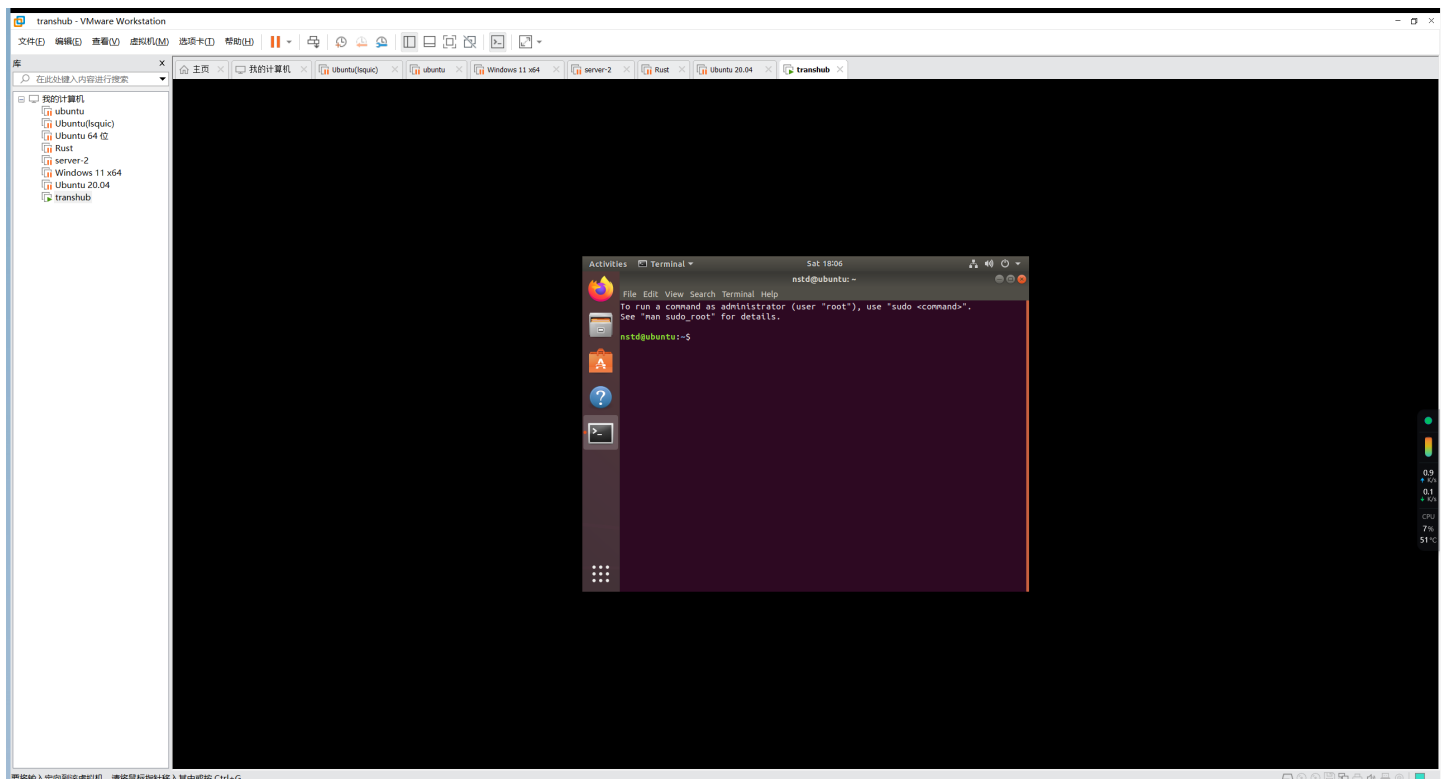
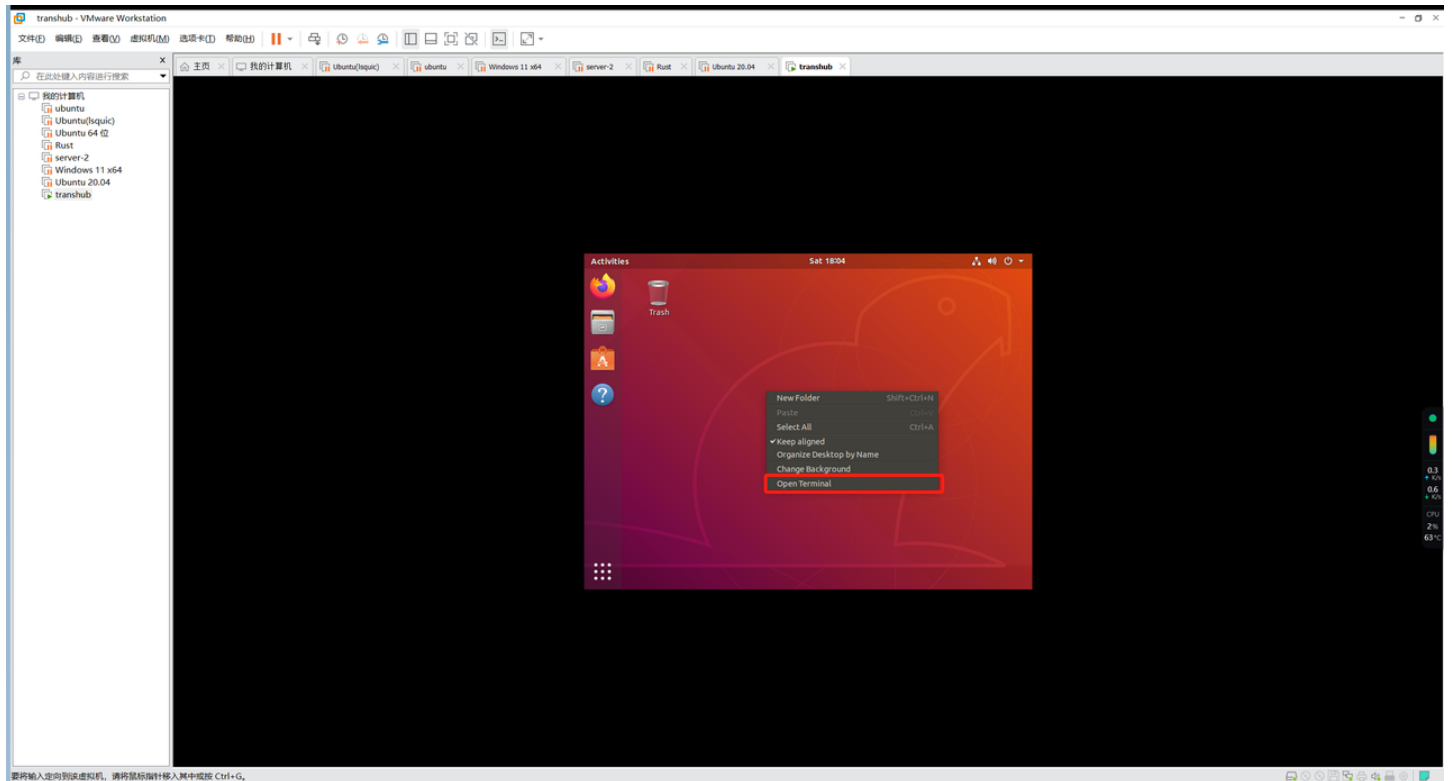
- 在其终端中运行如下代码，安装依赖项：

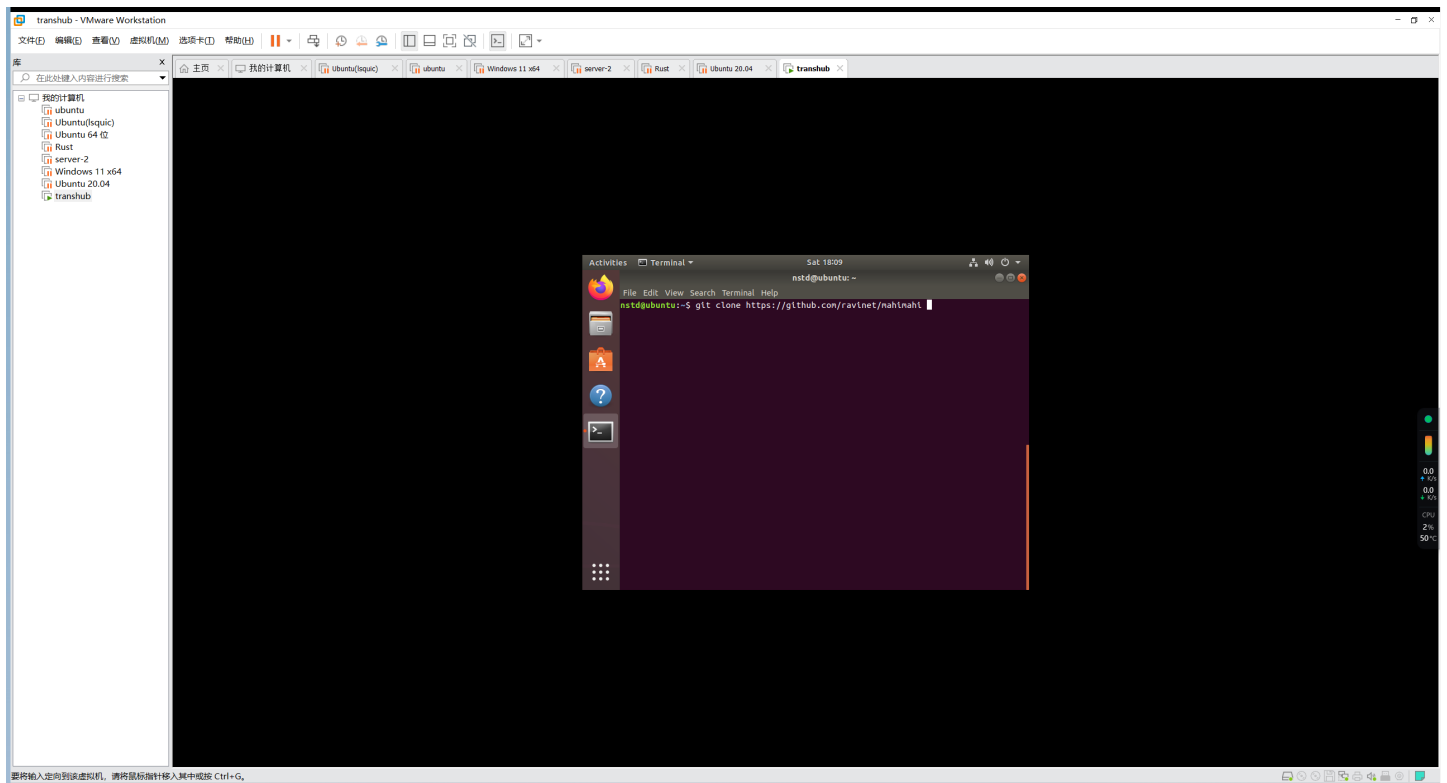
代码块

```
1 sudo apt-get install build-essential git debhelper autotools-dev dh-autoreconf  
iptables protobuf-compiler libprotobuf-dev pkg-config libssl-dev dnsmasq-base  
ssl-cert libxcb-present-dev libcairo2-dev libpango1.0-dev iproute2 apache2-dev
```

apache2-bin iptables dnsmasq-base gnuplot iproute2 apache2-api-20120211 libwww-perl

(这些依赖项在[mahimahi / debian / control](https://mahimahi.github.io/debian/control)中列出，另外还有一些用于比赛的依赖项)



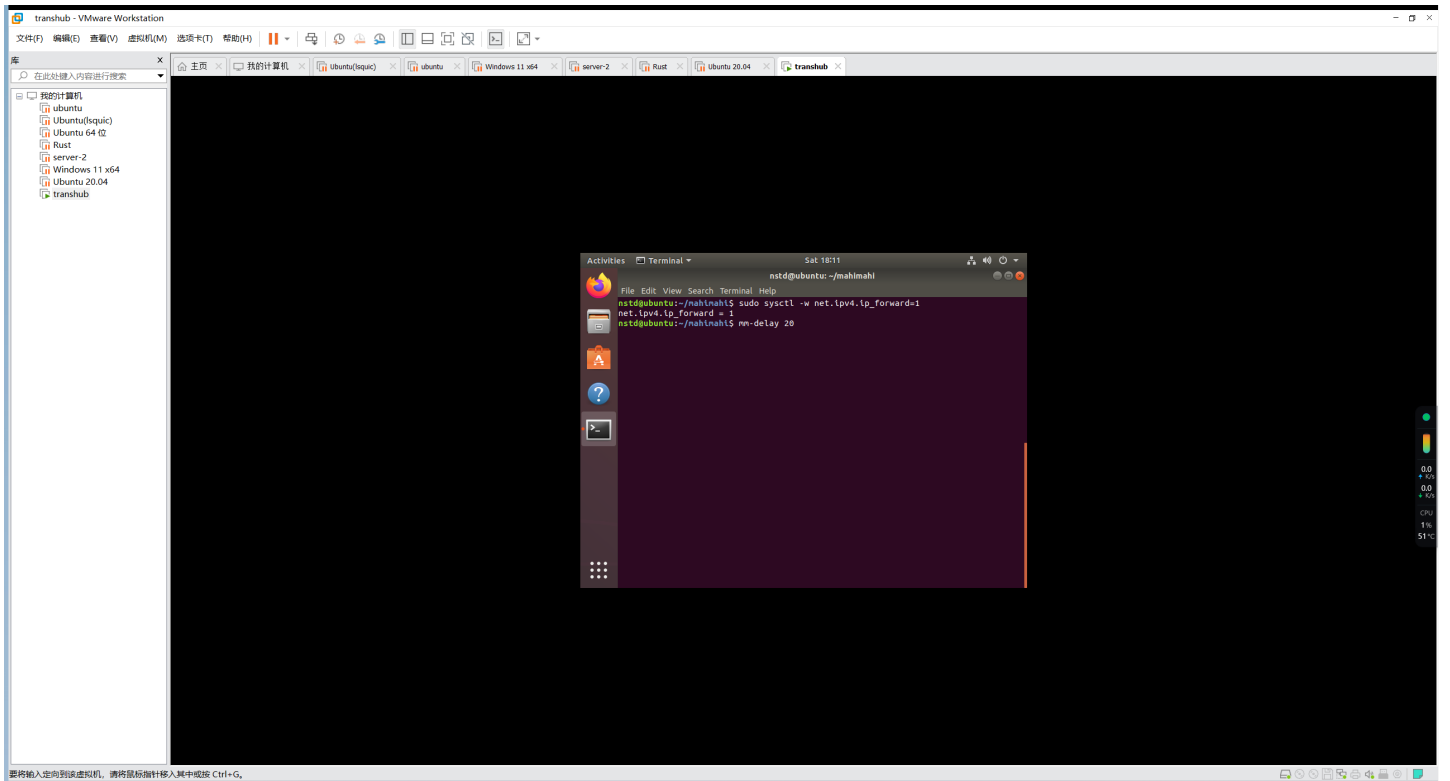
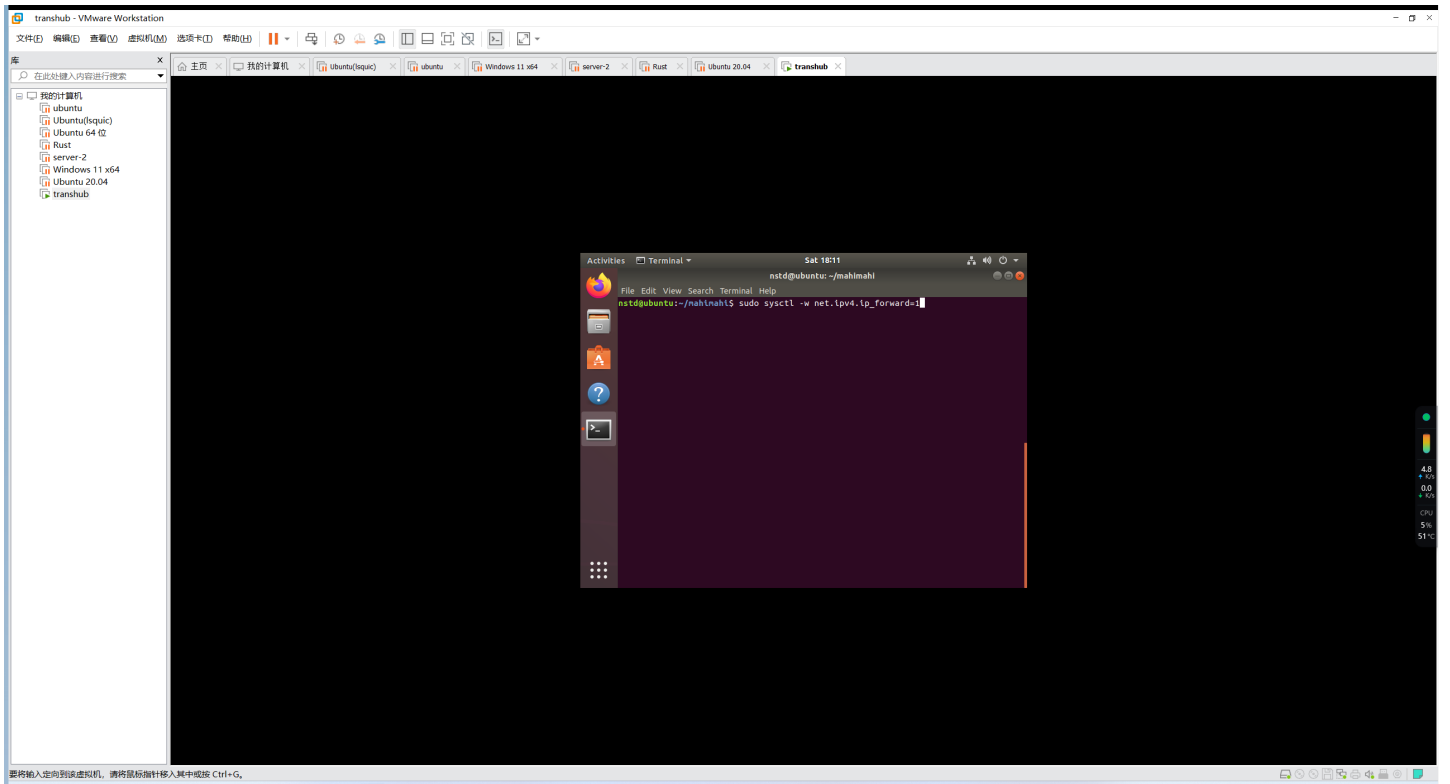


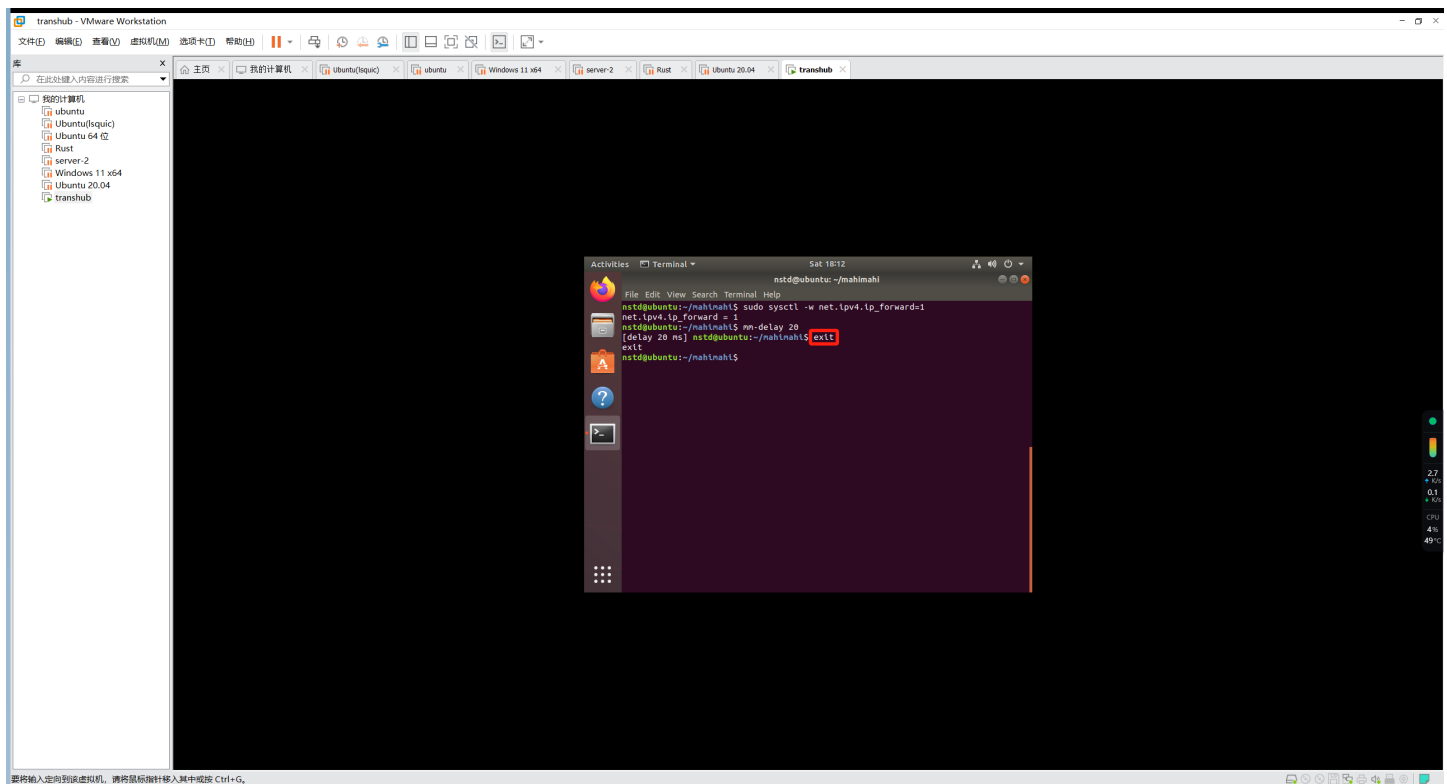
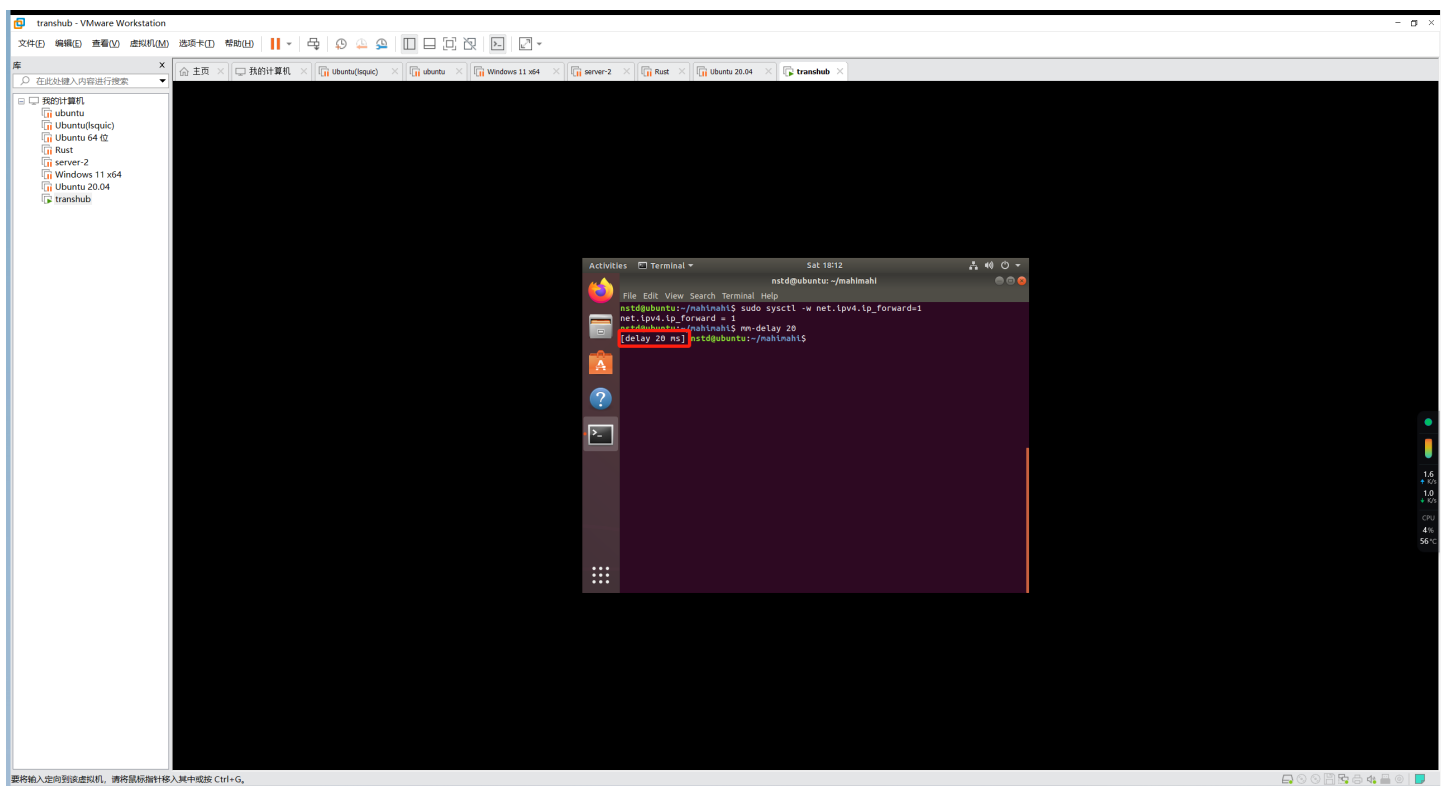
- 编译并安装mahimahi

代码块

```
1  cd mahimahi
2
3  # (用make工具编译mahimahi)
4  ./autogen.sh && ./configure && make
5
6  # 安装
7  sudo make install
8  sudo sysctl -w net.ipv4.ip_forward=1
9  # 进入mahimahi
10 mm-delay 20
11 # 退出mahimahi
12 exit
```

(若要检查是否安装成功, 可运行 `mm_delay 20` , 若出现delay 20ms字样, 说明安装成功)



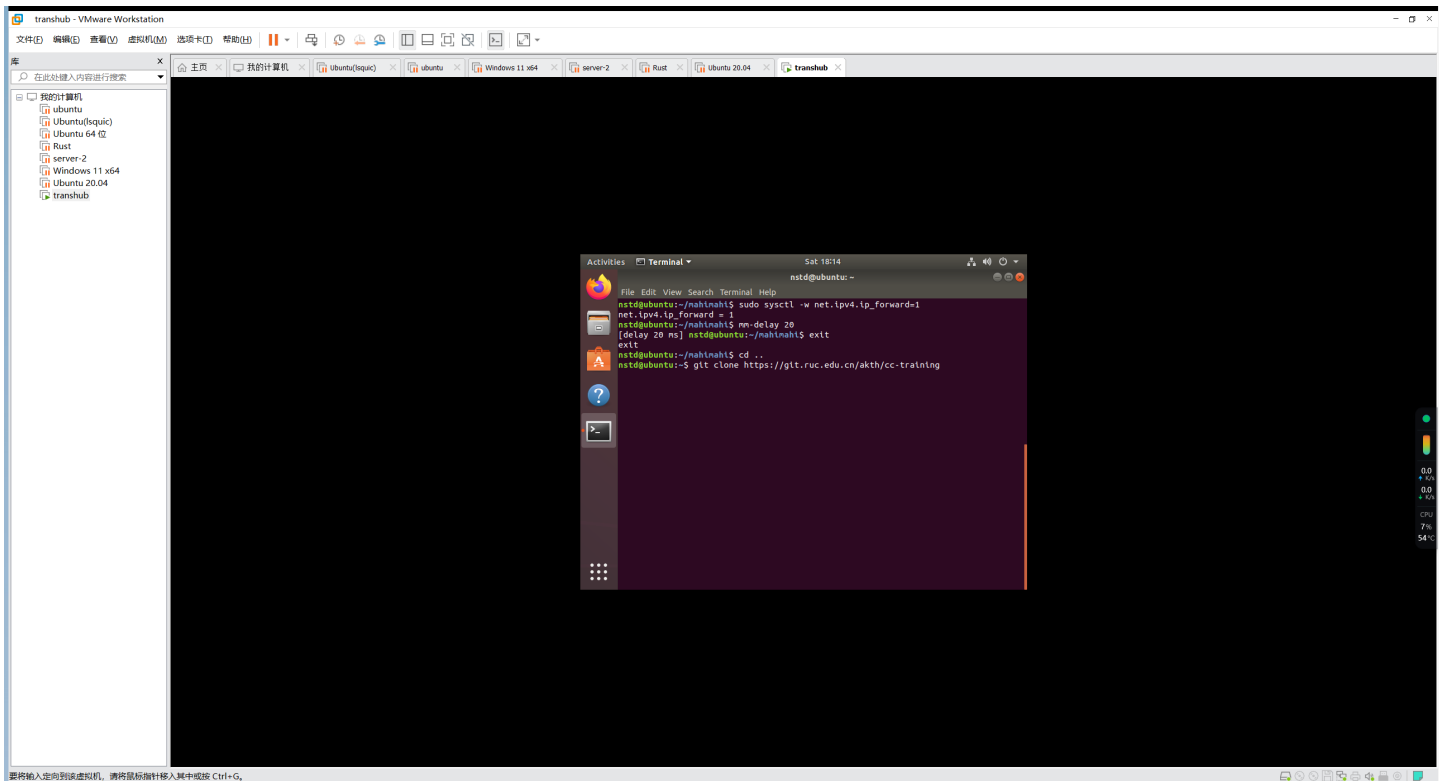
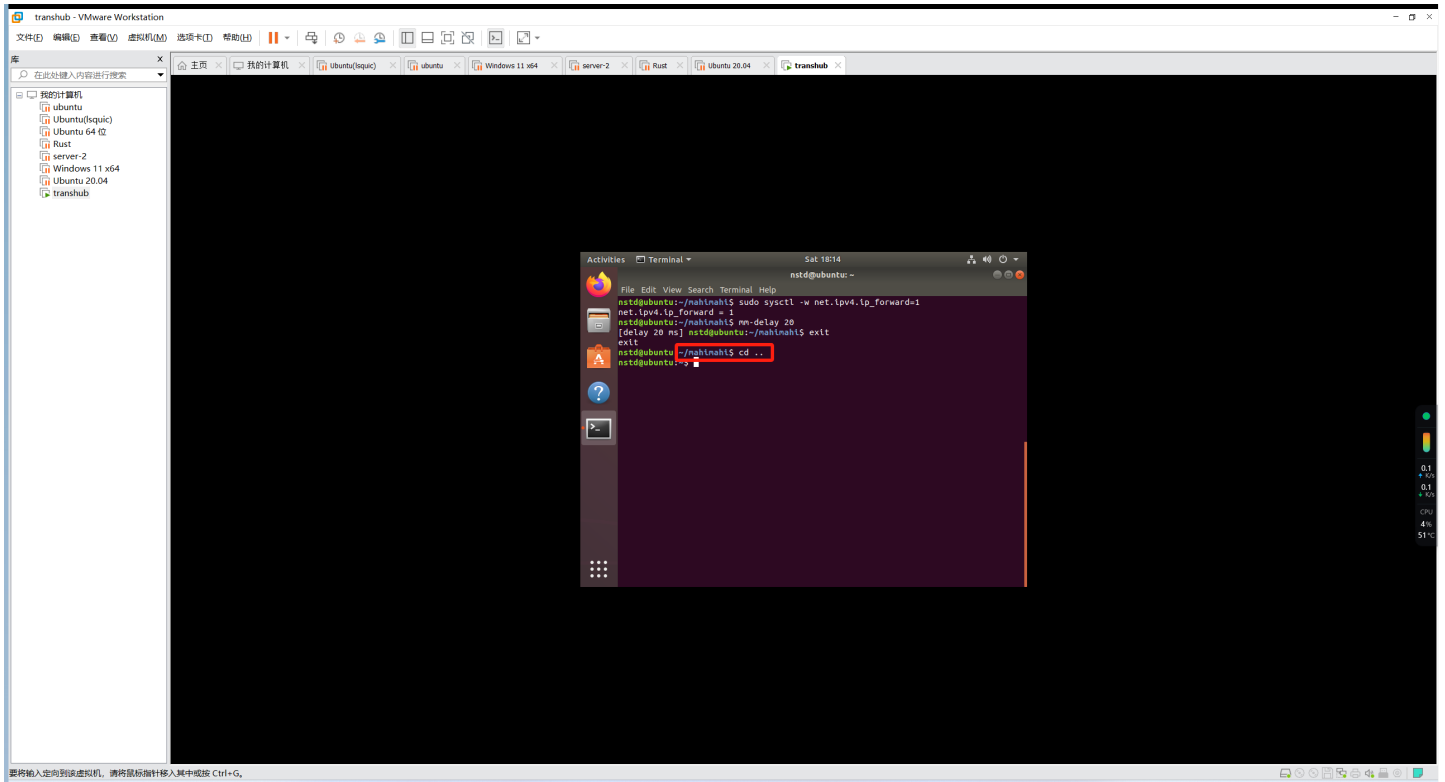


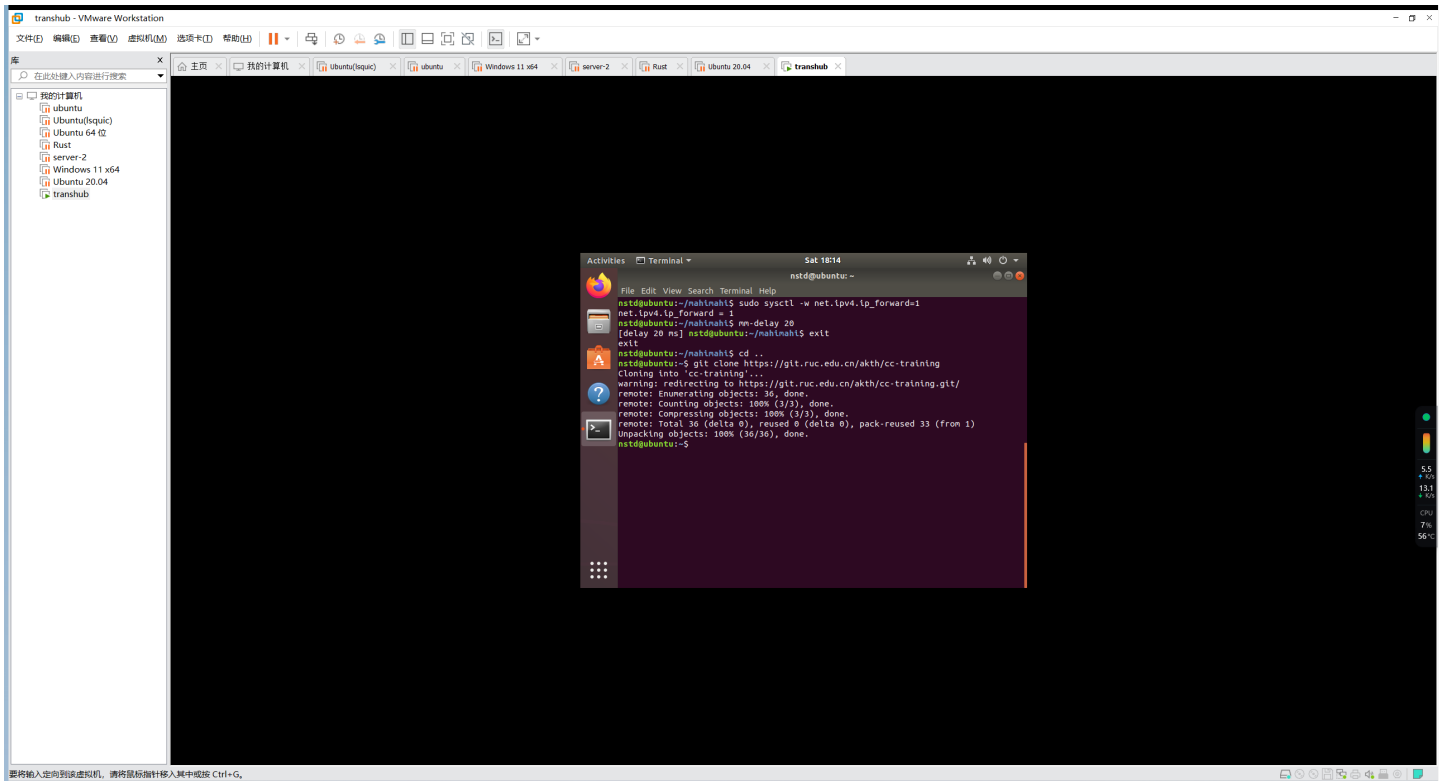
1.3. 编译Transhub代码

代码块

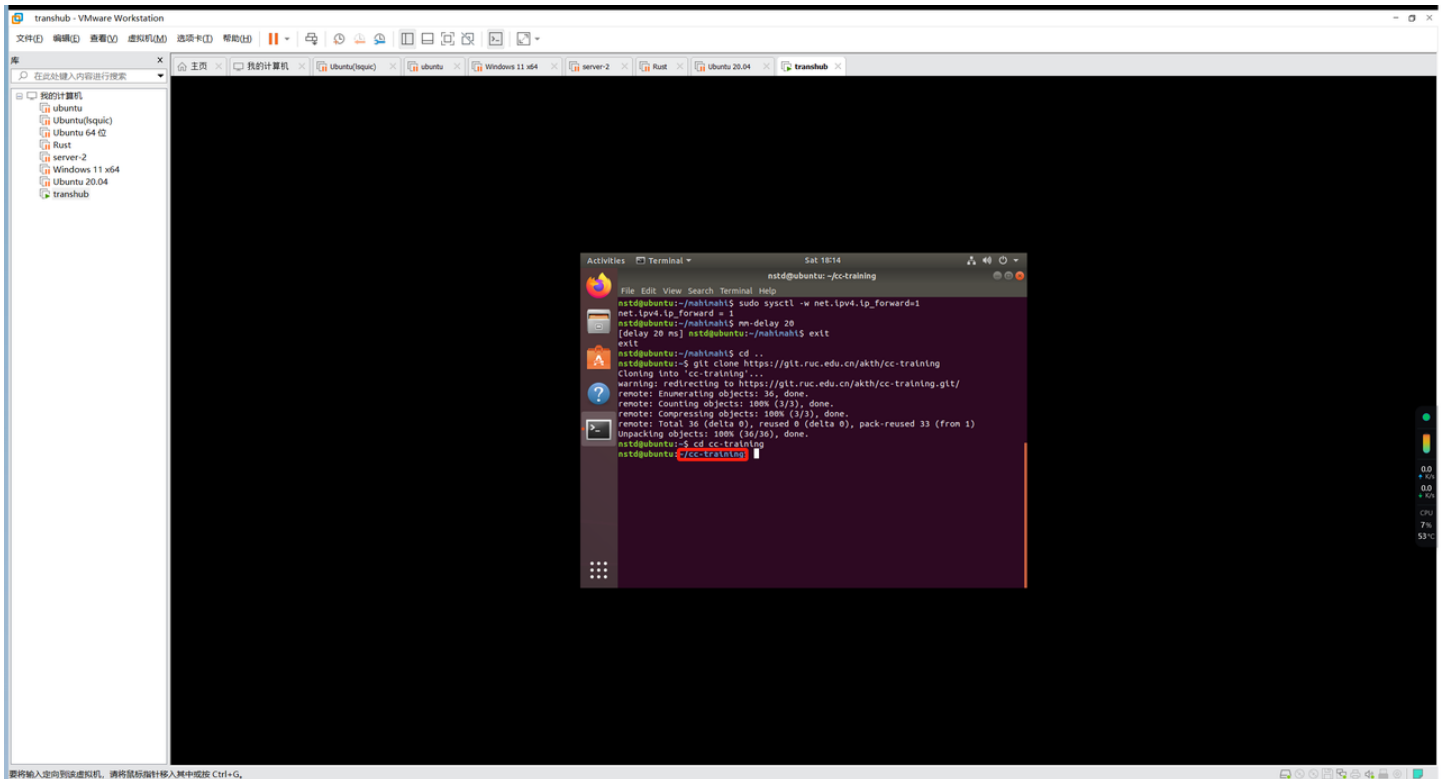
- 1 #退出mahimahi目录, 回到用户目录
- 2 `cd ..`
- 3 #下载transhub代码
- 4 `git clone https://git.ruc.edu.cn/akth/cc-training`

- 5 `cd cc-training`
- 6 `#按常规方式编译代码`
- 7 `./autogen.sh && ./configure && make`

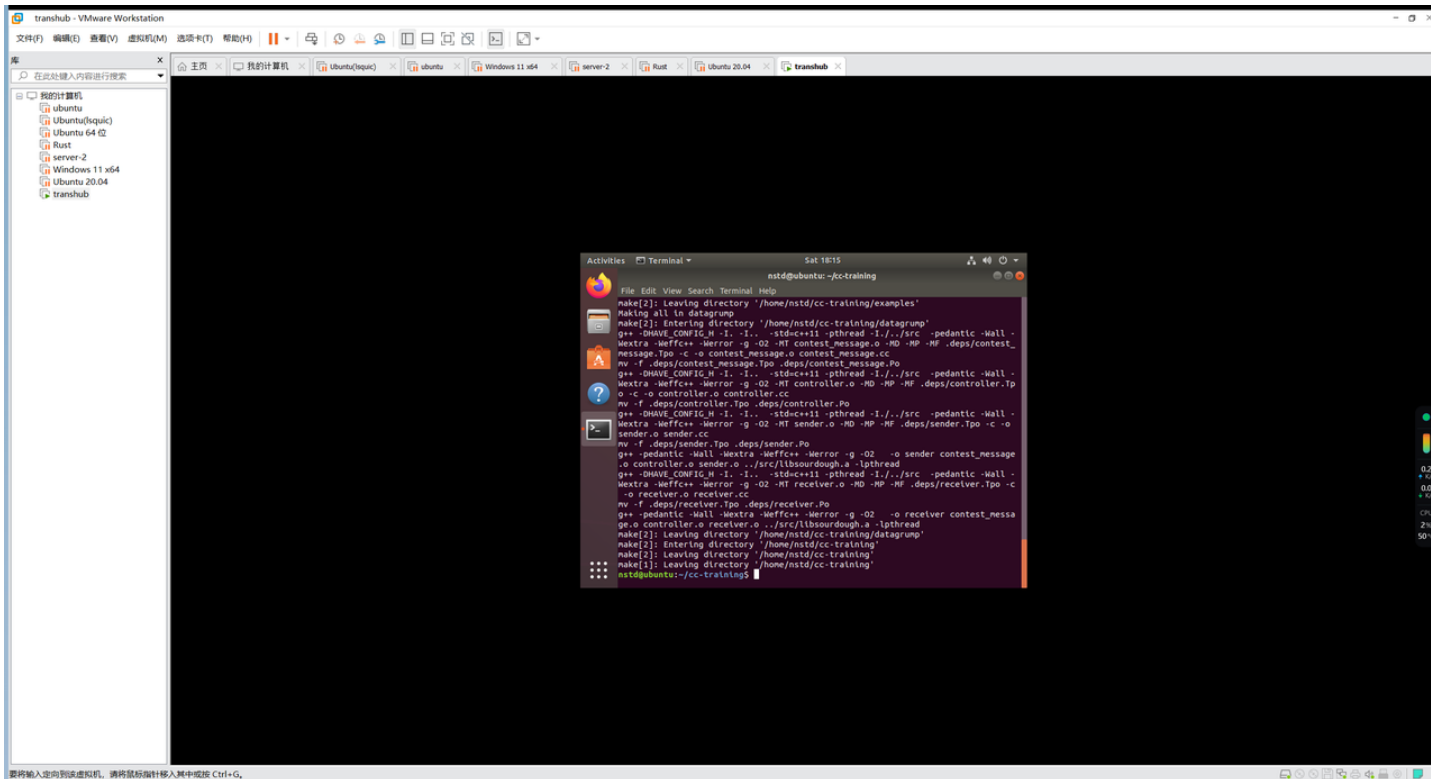
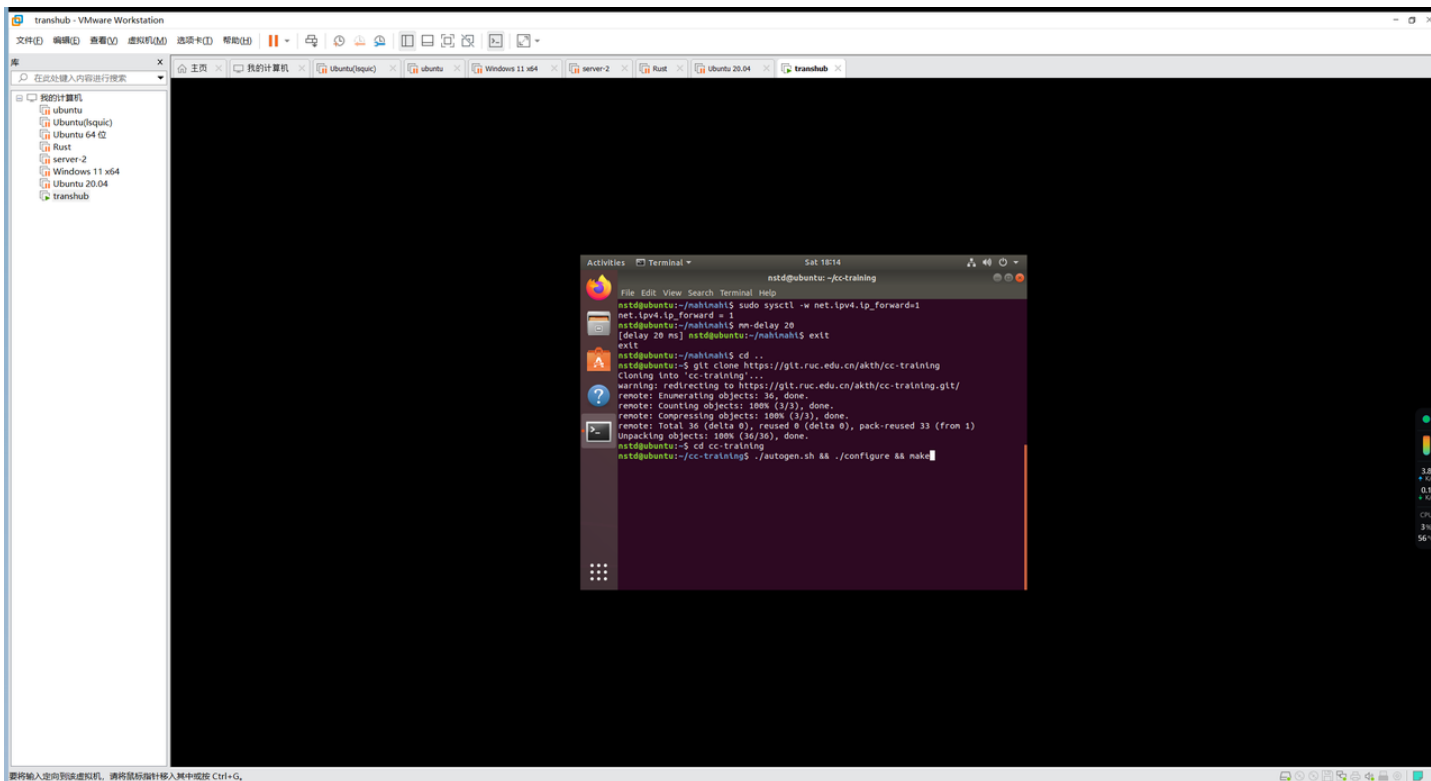




要将输入定向到该虚拟机，请将鼠标指针放入其中并按 Ctrl+G。



要将输入定向到该虚拟机，请将鼠标指针放入其中并按 Ctrl+G。



2. 方式二：使用已安装好Transhub的Docker镜像

★ MacOS系统推荐使用此方式

2.1部分仅限Mac，Linux/Windows平台参照对应教程安装好Docker后，后续方法（2.2-2.4）通用。

2.1. 安装Docker

Linux/Windows上如何安装使用Docker请自行查询相关教程，此处以Mac安装Docker为例。

Mac上可安装Docker Desktop或者Orbstack，这里以Orbstack为例。

OrbStack 是一款专为 MacOS 设计的轻量级容器和虚拟机管理工具，旨在为开发者提供快速、高效的容器化开发和测试环境。它支持 Docker 容器和 Linux 虚拟机，特别针对 Apple Silicon（M1/M2 等 ARM 芯片）进行了优化，能够无缝运行 x86 和 ARM 架构的容器。

访问<https://orbstack.dev/download> 下载并安装。安装好后，打开Orbstack完成初始化。

2.2. 下载并导入Transhub镜像

访问以下链接下载已配置好的transhub镜像：

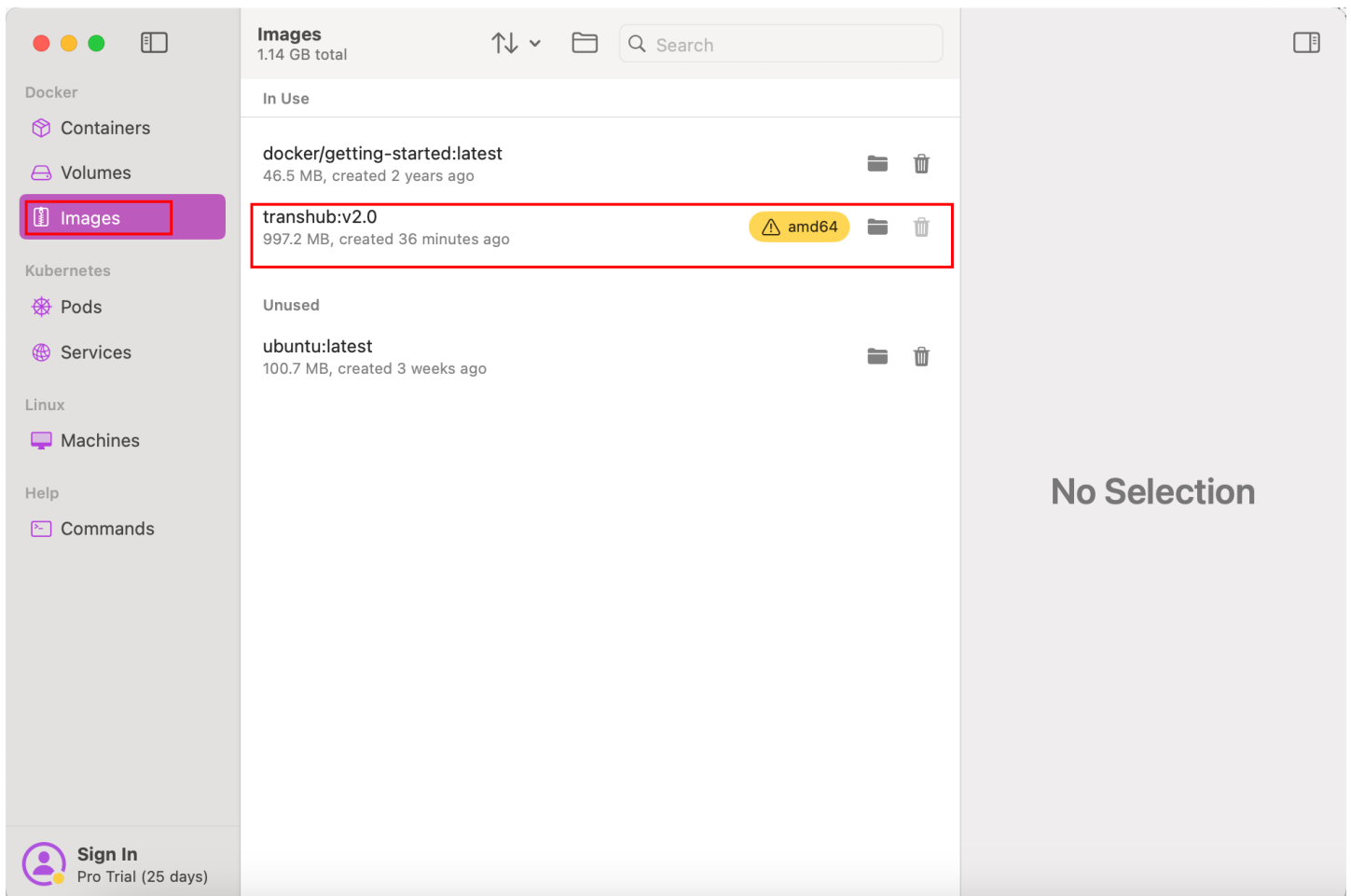
- 夸克网盘：链接：<https://pan.quark.cn/s/da8e2b2dfdd8> 提取码：ae6w
- 百度网盘：链接：<https://pan.baidu.com/s/10ltC9-XBsS0XKMpb7JBK0w?pwd=rt9x> 提取码: rt9x

在**镜像所在目录**运行终端，执行以下命令导入transhub镜像

代码块

```
1 docker load -i transhub.tar
```

导入成功后，可以在Orbstack看到该镜像：



2.3. 创建并运行Transhub容器

此步骤**仅需执行一次**，使用以下命令创建并运行transhub容器：

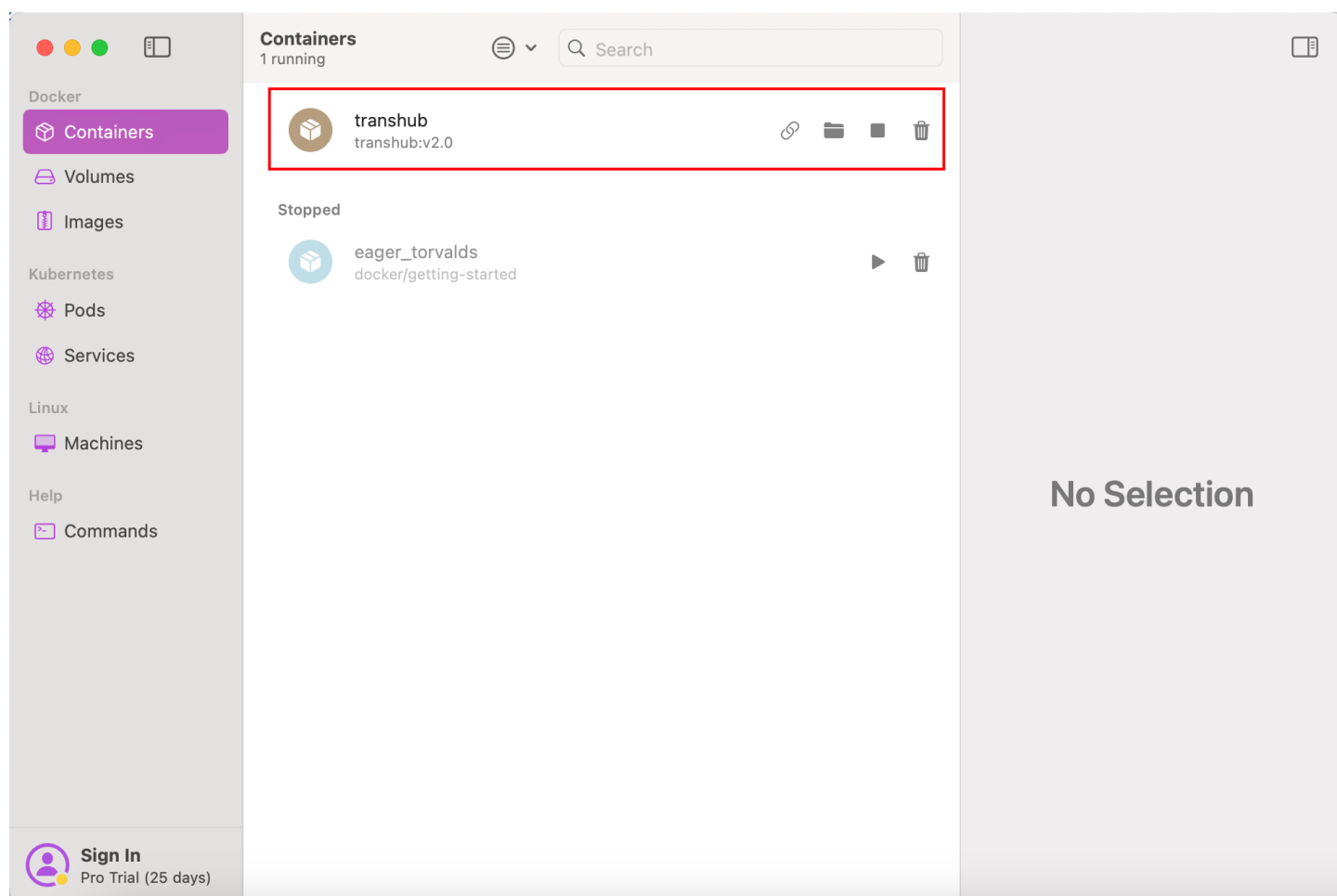
代码块

```
1 sudo docker run --platform linux/amd64 --privileged -itd --name transhub  
transhub:v2.0
```

！注意，在m芯片mac上，需要显式指定平台为 `linux/amd64`，强制运行 x86 架构的镜像

！注意，如需要使用 `-v` 命令将主机上的目录挂载到容器目录，请勿覆盖容器内的 `/home/none_root/transhub` 目录。因为该目录下已包含配置好的代码文件，如映射后，将被主机目录覆盖。即如需使用 `-v ~/Downloads/transhub:/home/none_root/transhub`，`:` 后不得接 `/home/none_root/transhub`

运行成功后，可以在Orbstack中看到其已处于运行状态：



2.4. 进入Transhub容器

- 首先，确保容器已经启动并正在运行。在执行完2.3步后，容器已经处于运行状态。后续若需再次使用容器，则无需再重复执行2.3步骤，可使用以下方法来启动容器。

`docker run` 命令通常用于第一次启动一个新的容器。当你执行 `docker run` 时，Docker会执行两个步骤：首先，它会从指定的镜像创建一个新的容器（相当于执行了 `docker create` 命令），然后它会启动这个容器，使其变成一个运行中的容器（这一步相当于执行了 `docker start` 命令）。因此，`docker run` 是一个组合命令，它不仅创建容器，还会立即启动容器。如果在系统上找不到指定的镜像，`docker run` 甚至会尝试从Docker Hub拉取镜像。相比之下，`docker start` 命令用于启动一个已经存在的容器。如果你之前已经创建了一个容器（无论是通过 `docker create` 创建的，还是之前用 `docker run` 创建并启动过的），你可以使用 `docker start` 来重新启动这个容器。这意味着，使用 `docker start` 时，你必须知道要启动的容器的ID或名称。

可以运行以下命令查看容器状态：

代码块

```
1 docker ps
```

如果容器正在运行，你会看到类似以下的输出：

代码块

	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
1	NAMES					
2	eecfe24288b1	transhub:v2.0	"/bin/bash"	5 minutes ago	Up 5 minutes	
	transhub					

如果容器没有运行，可以使用以下命令启动它：

代码块

```
1 docker start transhub
```

- 使用 `docker exec` 命令进入容器的 Shell，后续即可在终端中操作该容器

代码块

```
1 docker exec -it transhub /bin/bash
```

- 接下来可进入容器的目录 `/home/none_root/transhub`，可以看到已安装好的mahimahi和transhub文件：

代码块

```
1 cd home/none_root/transhub/  
2 ls
```

```
root@12be595786c5: /home/none_root/transhub/cc-training/datagrump — docker exec -it transhub /bin/bash — 80x24  
~/Downloads — -zsh  ...one_root/transhub — -zsh  ~/Downloads — -zsh  ...c -it transhub /bin/bash +  
[zhujiuxiang@JIUXIANGZHU-MC1 Downloads % docker exec -it transhub /bin/bash  
root@12be595786c5:/# cd home/none_root/transhub/  
root@12be595786c5:/home/none_root/transhub# ls  
cc-training  mahimahi  
root@12be595786c5:/home/none_root/transhub# cd cc-training/  
root@12be595786c5:/home/none_root/transhub/cc-training# ls  
Makefile      aclocal.m4      config.h      configure      examples      stamp-h1  
Makefile.am   autogen.sh      config.h.in   configure.ac   install-sh  
Makefile.in   autom4te.cache  config.log    datagrump      missing  
README.md     compile         config.status depcomp        src  
root@12be595786c5:/home/none_root/transhub/cc-training# cd datagrump/  
root@12be595786c5:/home/none_root/transhub/cc-training/datagrump# ls  
Makefile      contest_message.cc  controller.cc  receiver.cc  sender.cc  
Makefile.am   contest_message.hh  controller.hh  receiver.o   sender.o  
Makefile.in   contest_message.o   controller.o   run-contest  
a.svg         contest_uplink_log  receiver      sender
```

（二）运行实验测试

运行 `run-contest` 开始测试后，程序将模拟VerizonLTE连接大约两分钟，请耐心等待，运行完成时会生成日志。

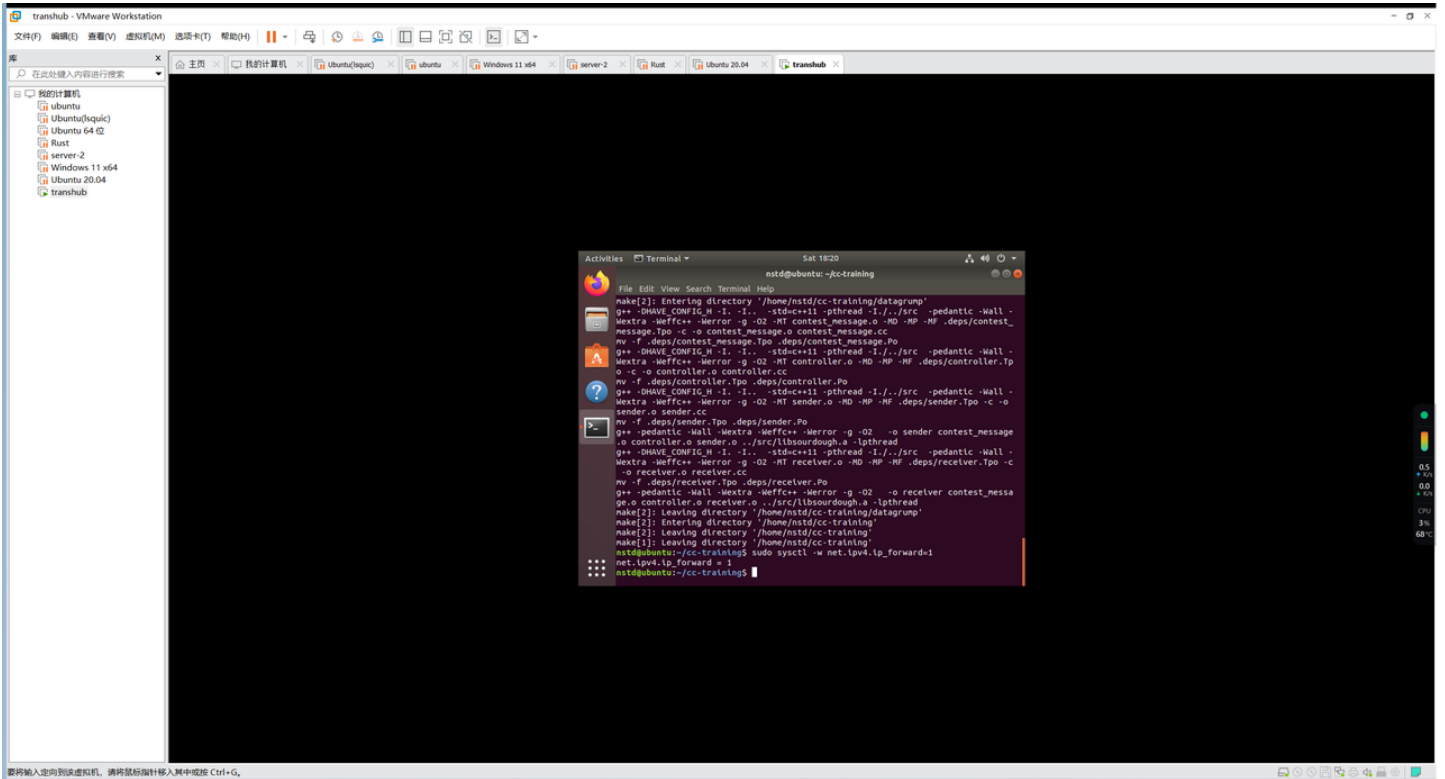
💡 `run-contest` 工具负责将测试文件通过模拟Verizon downlink生成日志，并使用mahimahi仿真，提供传输过程中的性能信息，并显示queueing delay和throughput随时间变化的折线图。

2.1. 通过方式一安装的测试过程

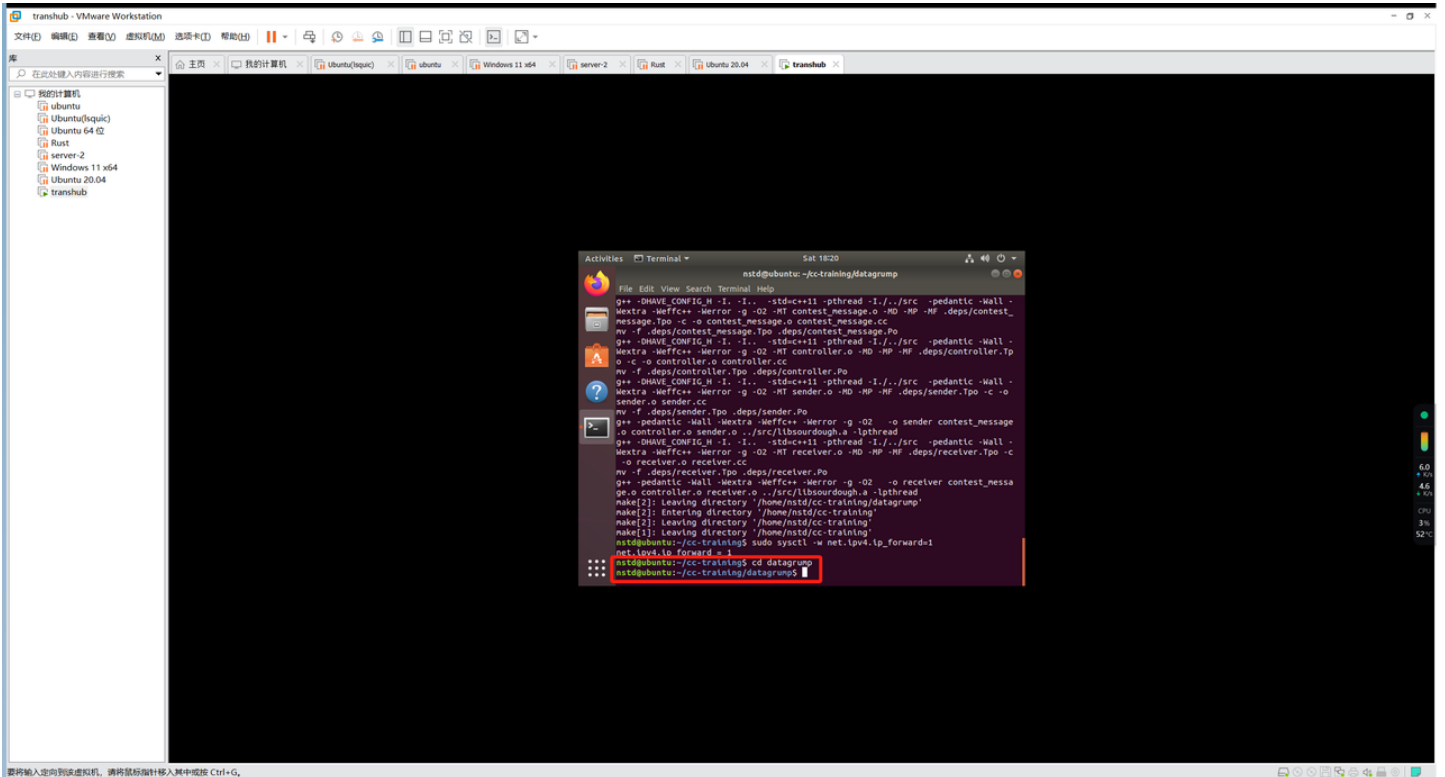
- 运行 `run-contest` 开始实验

代码块

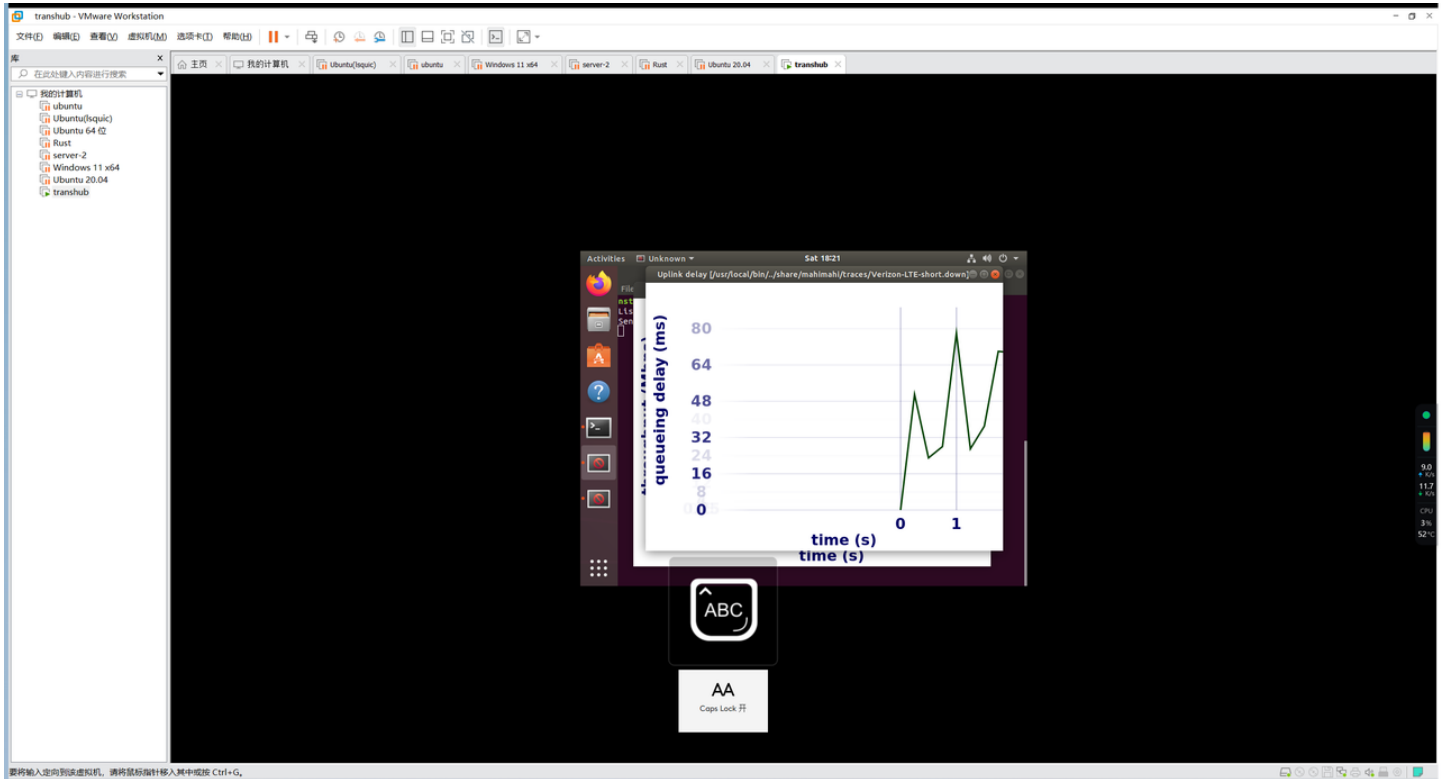
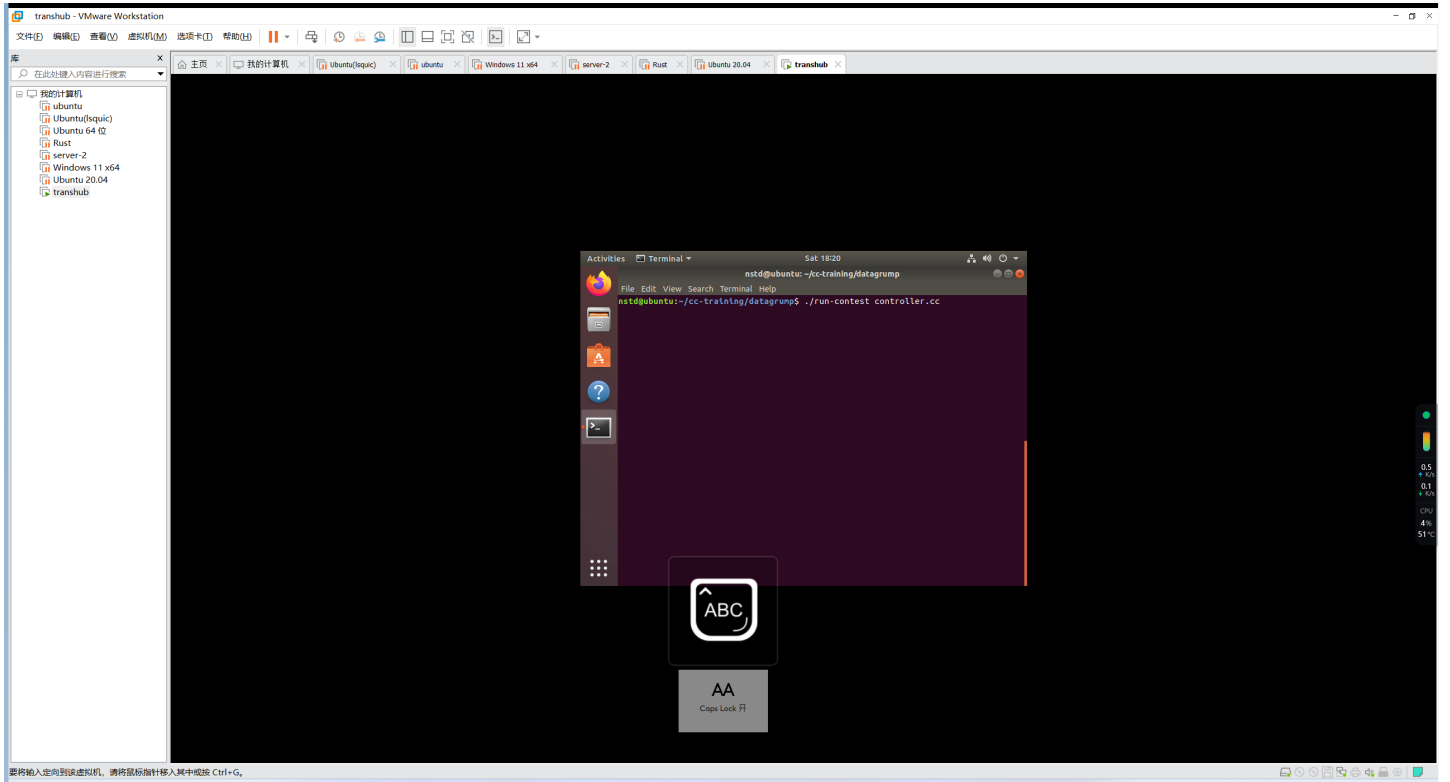
```
1 sudo sysctl -w net.ipv4.ip_forward=1 # (必须启用Linux的IP转发才能使mahimahi工作)  
2 cd datagrump # (前提已经在cc-training目录下)  
3 ./run-contest [scheme_name] # (scheme_name就是你给自己文件命名的名称，第一次使用时可将[scheme_name] 替换为controller.cc)
```

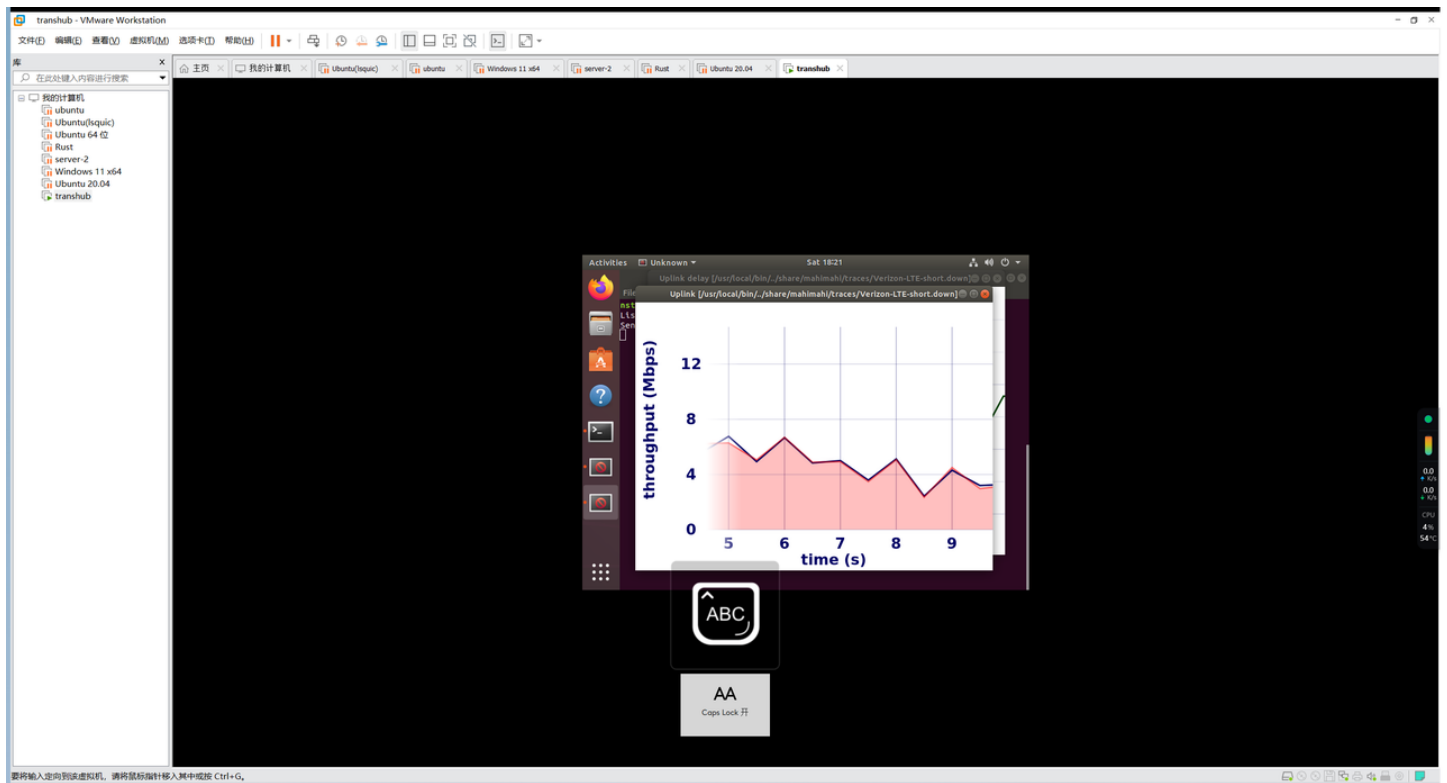


要将输入定向到该虚拟机，请将鼠标指针移入其中并按 Ctrl+G。



要将输入定向到该虚拟机，请将鼠标指针移入其中并按 Ctrl+G。

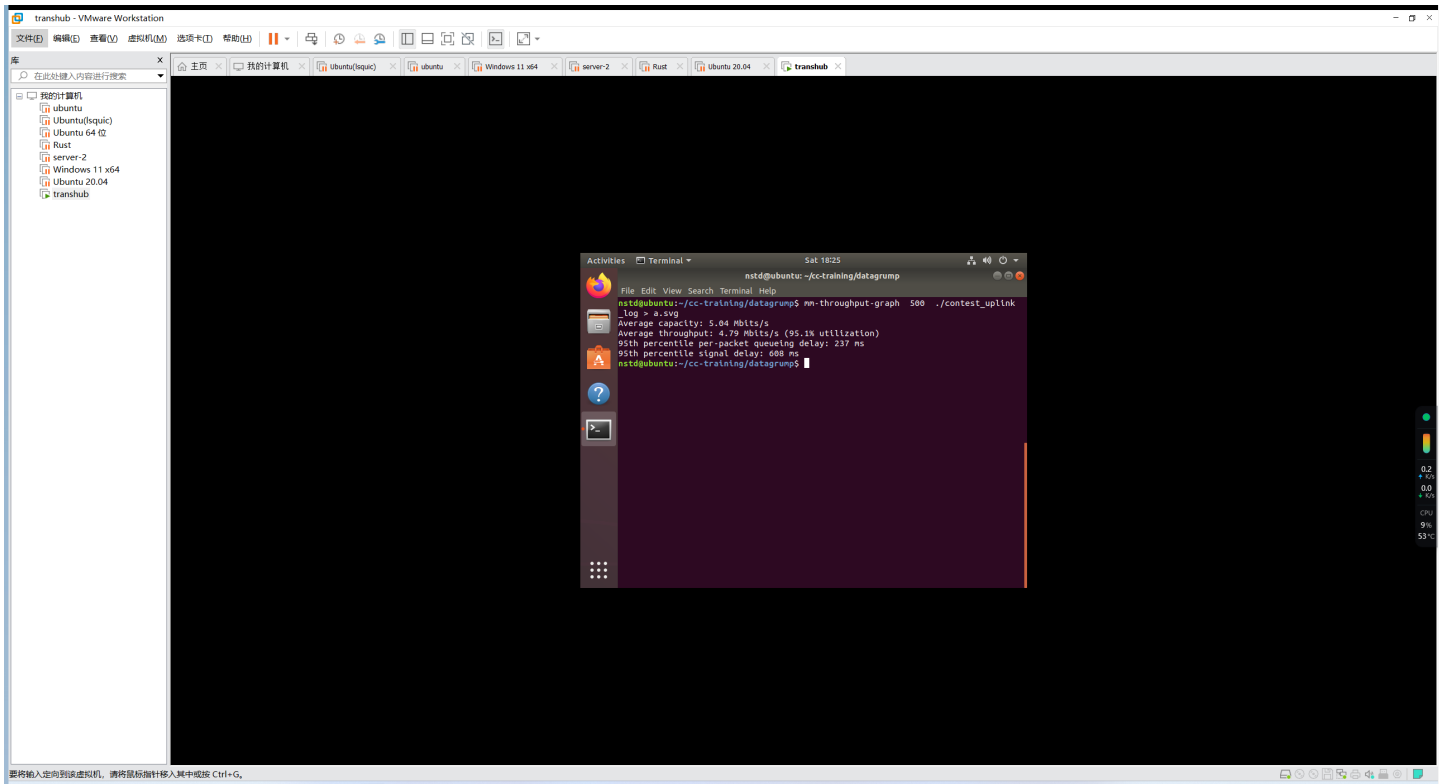




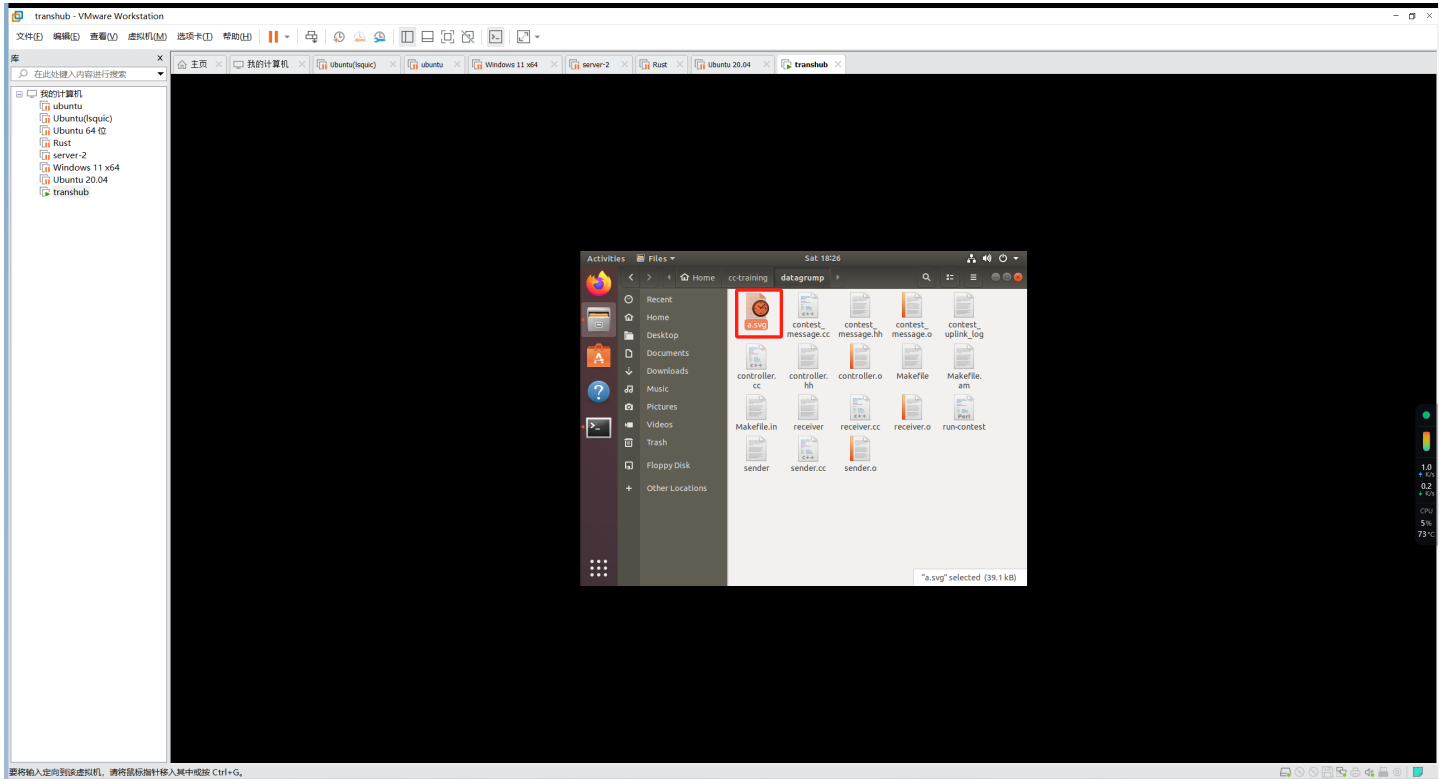
- 使用如下命令获取实验结果

代码块

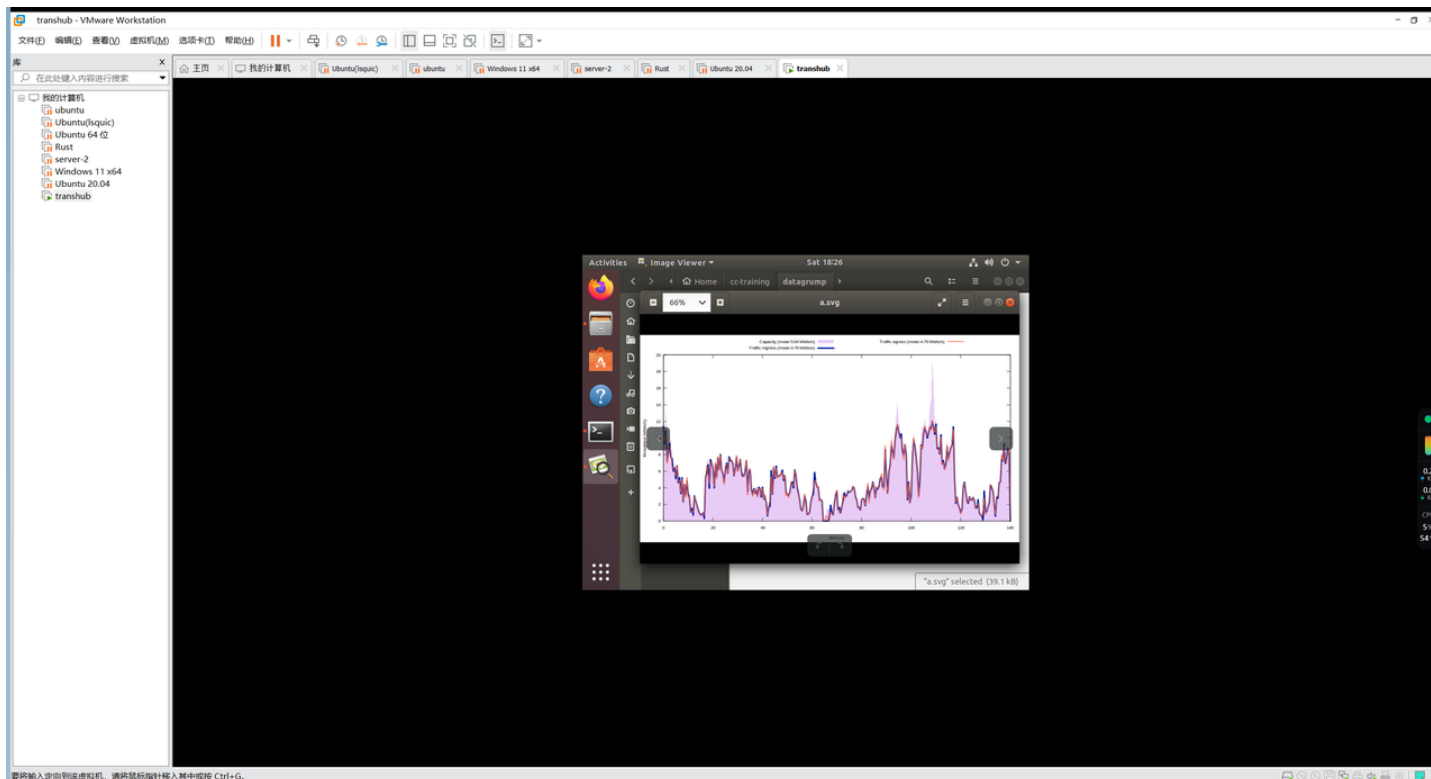
```
1 mm-throughput-graph 500 ./contest_uplink_log > a.svg # (分析日志, 输出吞吐等信息, 得到svg图)
2
3 # 得到以下结果
4 Average capacity:平均容量
5 Average throughput:平均吞吐量
6 95(th) percentile per-packet queueing delay:95%排队延迟
7 95(th) percentile signal delay:95%信号延迟
```

要将输入定向到该虚拟机，请将鼠标指针放入其中或按 Ctrl+G。



要将输入定向到该虚拟机，请将鼠标指针放入其中或按 Ctrl+G。



2.2. 通过方式二安装的测试过程

！注意，在容器中运行时，运行 `run-contest` 需要在非root用户下执行，而开启linux的IP转发以及编译源代码需要在root用户下。使用 `su none_root` 切换为非root用户，使用 `exit` 切换回root用户。在使用过程中请注意在两种用户间灵活切换。

- 运行 `run-contest` 开始实验

代码块

```
1  sysctl -w net.ipv4.ip_forward=1 # (必须启用Linux的IP转发才能使mahimahi工作，此操作需在root用户下执行)
2  cd datagrump # (前提已经在cc-training目录下)
3  su none_root # (切换为非root用户。若是想再切换回root，只需要输入exit然后按下回车即可)
4  ./run-contest [scheme_name] # (scheme_name就是你给自己文件命名的名称，第一次使用时可以将[scheme_name] 替换为controller.cc)
```

执行命令时，若如下方提示 `please run as non-root`，请使用命令 `su none_root` 切换到 `non_root` 用户。

代码块

```
1  root@12be595786c5:/home/none_root/transhub/cc-training/datagrump# ./run-contest controller.cc
2  Listening on :::9090
3  Died on std::runtime_error: mm-delay: please run as non-root
```

由非root用户切换回root用户示意图：

```
none_root@12be595786c5:~/transhub/cc-training/datagrump$ sudo sysctl -w net.ipv4
.ip_forward=1
bash: sudo: command not found
none_root@12be595786c5:~/transhub/cc-training/datagrump$ exit
exit
root@12be595786c5:/home/none_root/transhub/cc-training/datagrump# sysctl -w net.
ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@12be595786c5:/home/none_root/transhub/cc-training/datagrump#
```

非root用户下无法执行

切换回root用户

- 使用如下命令获取实验结果

代码块

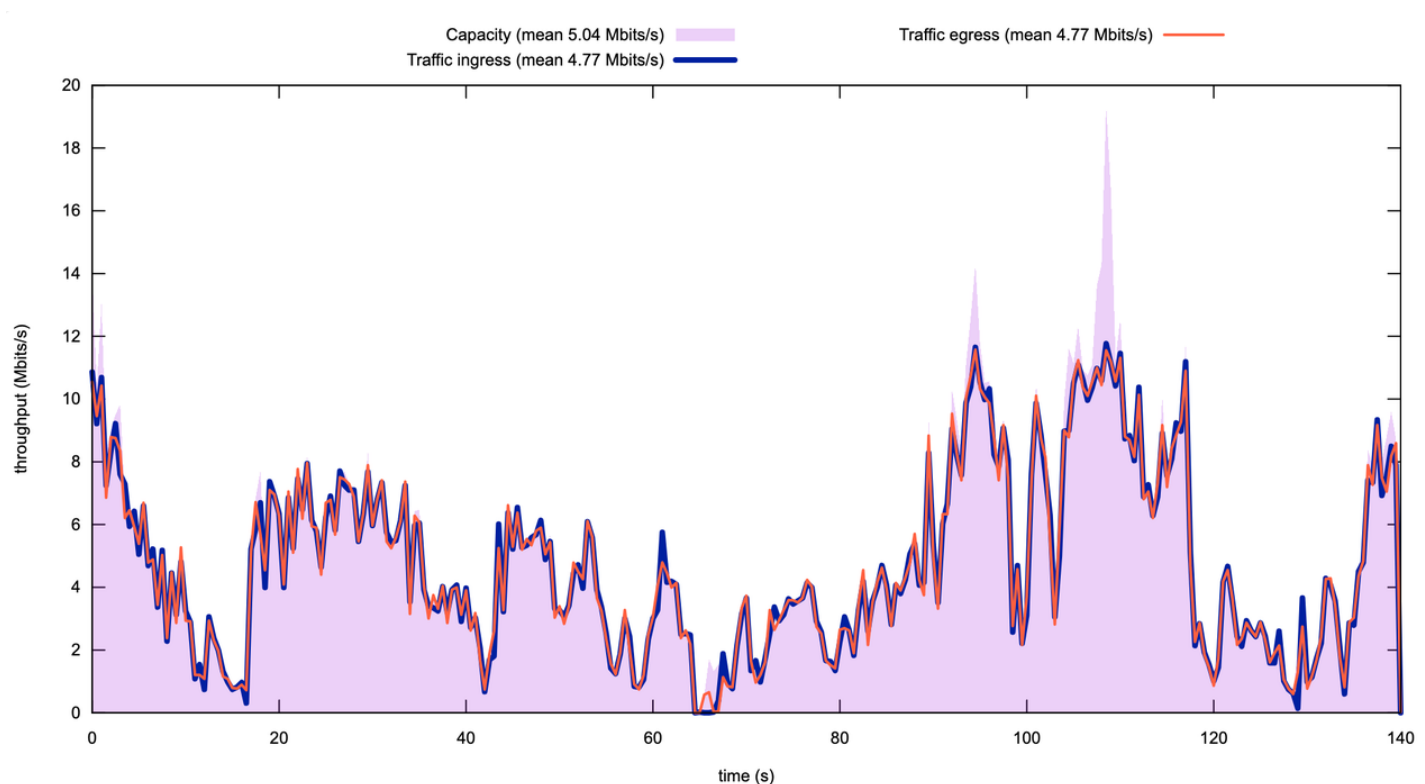
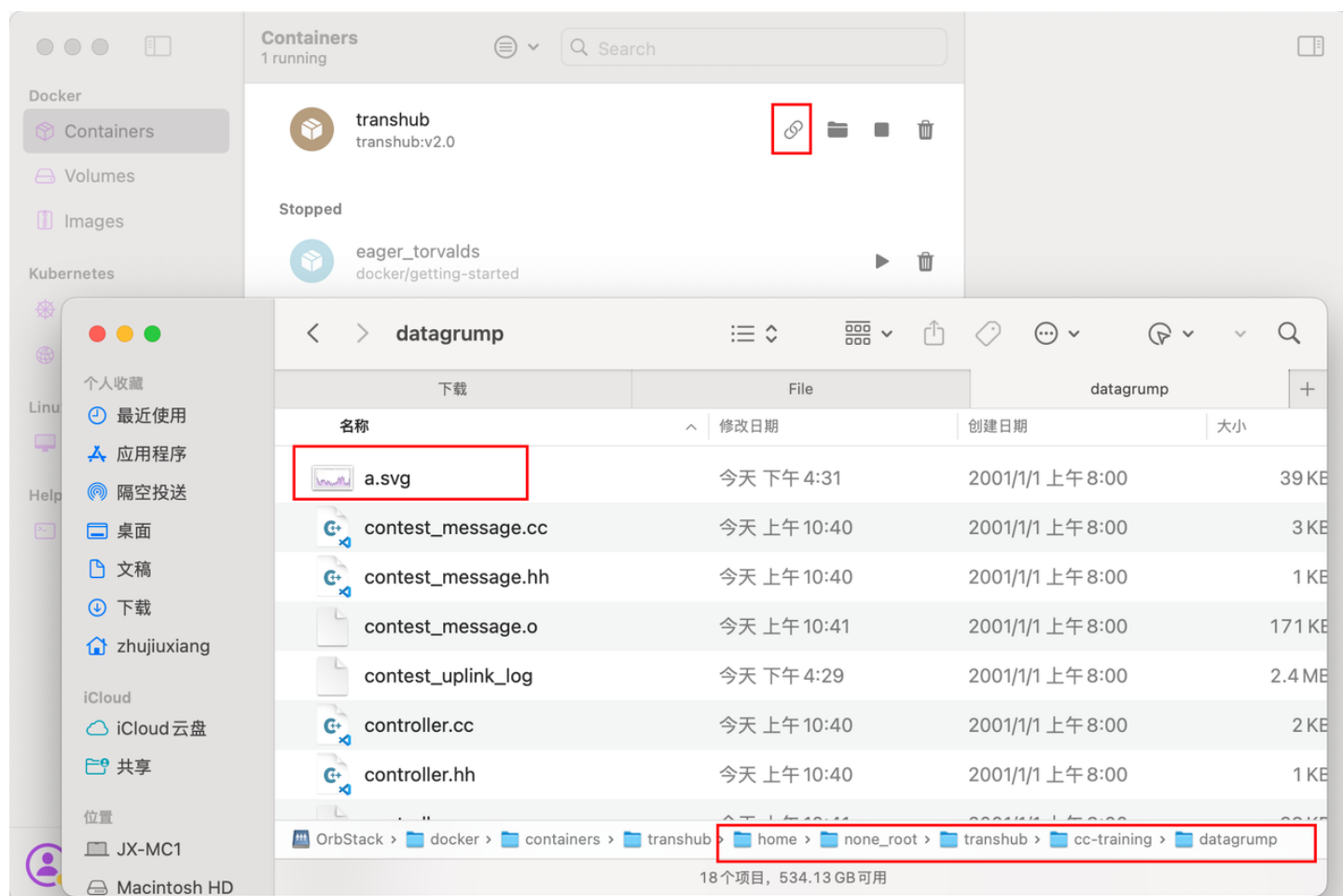
```
1 mm-throughput-graph 500 ./contest_uplink_log > a.svg # (分析日志, 输出吞吐等信
 息, 得到svg图)
2
3 # 得到以下结果
4 Average capacity:平均容量
5 Average throughput:平均吞吐量
6 95(th) percentile per-packet queueing delay:95%排队延迟
7 95(th) percentile signal delay:95%信号延迟
```

下载 — root@12be595786c5: /home/none_root/transhub/cc-training/datagrump — docker exec -it transhub /bin/bash — 80x12

~/Downloads — -zsh ...one_root/transhub — -zsh ~/Downloads — -zsh ...c -it transhub /bin/bash +

```
none_root@12be595786c5:~/transhub/cc-training/datagrump$ mm-throughput-graph 50
0 ./contest_uplink_log > a.svg
Average capacity: 5.04 Mbits/s
Average throughput: 4.77 Mbits/s (94.6% utilization)
95th percentile per-packet queueing delay: 236 ms
95th percentile signal delay: 606 ms
none_root@12be595786c5:~/transhub/cc-training/datagrump$ ls
Makefile      contest_message.cc  controller.cc  receiver.cc  sender.cc
Makefile.am   contest_message.hh  controller.hh  receiver.o   sender.o
Makefile.in   contest_message.o   controller.o   run-contest
a.svg         contest_uplink_log  receiver      sender
none_root@12be595786c5:~/transhub/cc-training/datagrump$
```

Mac可通过以下方式找到容器内文件：



🎉 Congratulations! 至此，你已成功安装transhub，接下来请按照后续任务书要求，完成相应任务。

（三）系统拥塞控制算法配置（*可选）

本节内容介绍了如何查看并修改系统拥塞控制算法，属于**拓展阅读**部分。

Linux系统中可选的拥塞控制算法包括 `reno`、`cubic`、`bbr` 等。本节以配置 `bbr` 算法为例：

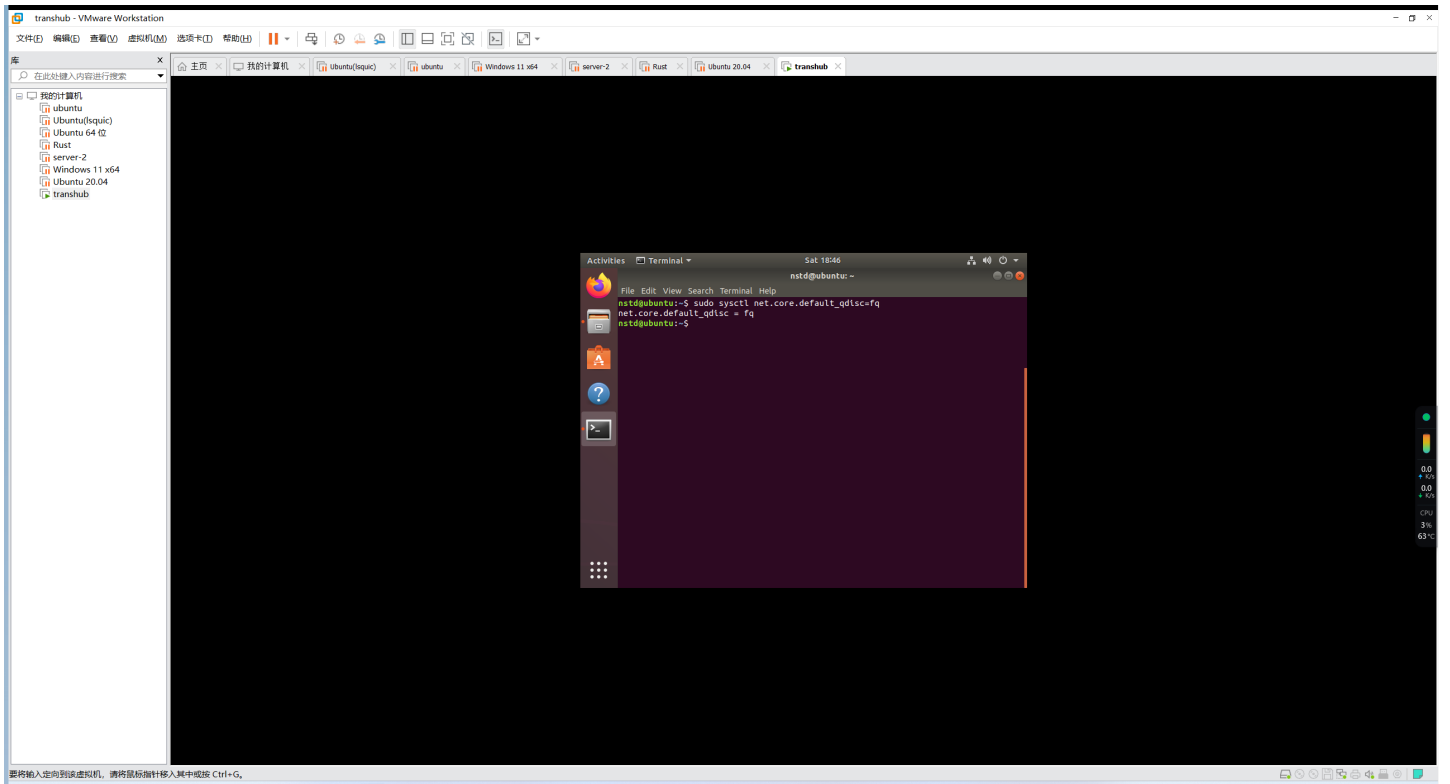
❗ 配置不同的拥塞控制算法会对传输结果产生不同的影响，可做相关实验验证（此处所指**实验并非**本transhub中的 `run-contest` 实验，`run-contest` 的拥塞控制算法为 `controller.cc`，与系统拥塞控制算法无关。如需验证系统拥塞控制算法，可使用 `iperf` 等工具来做相关实验）。

3.1 通过方式一安装的操作过程

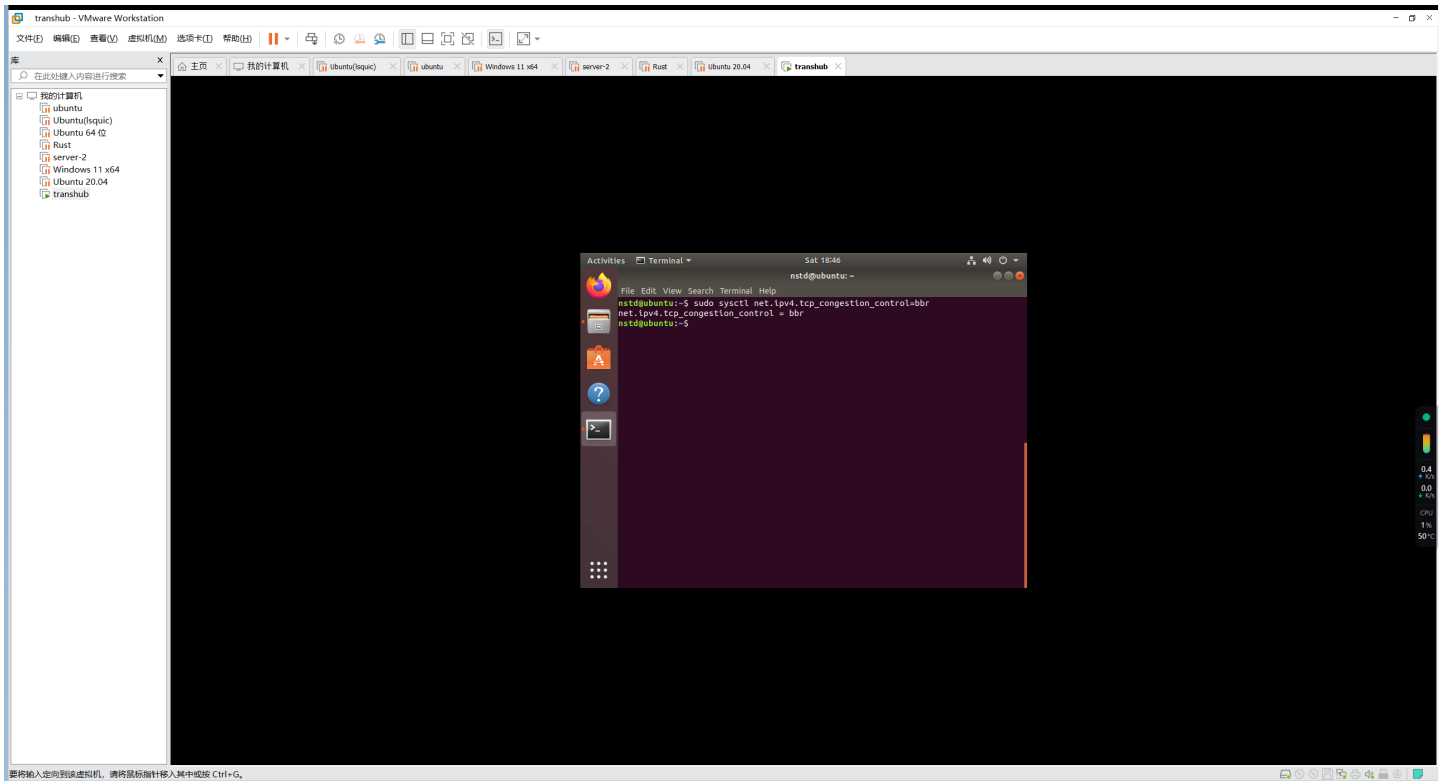
💡 在Linux主机上启用BBR算法

代码块

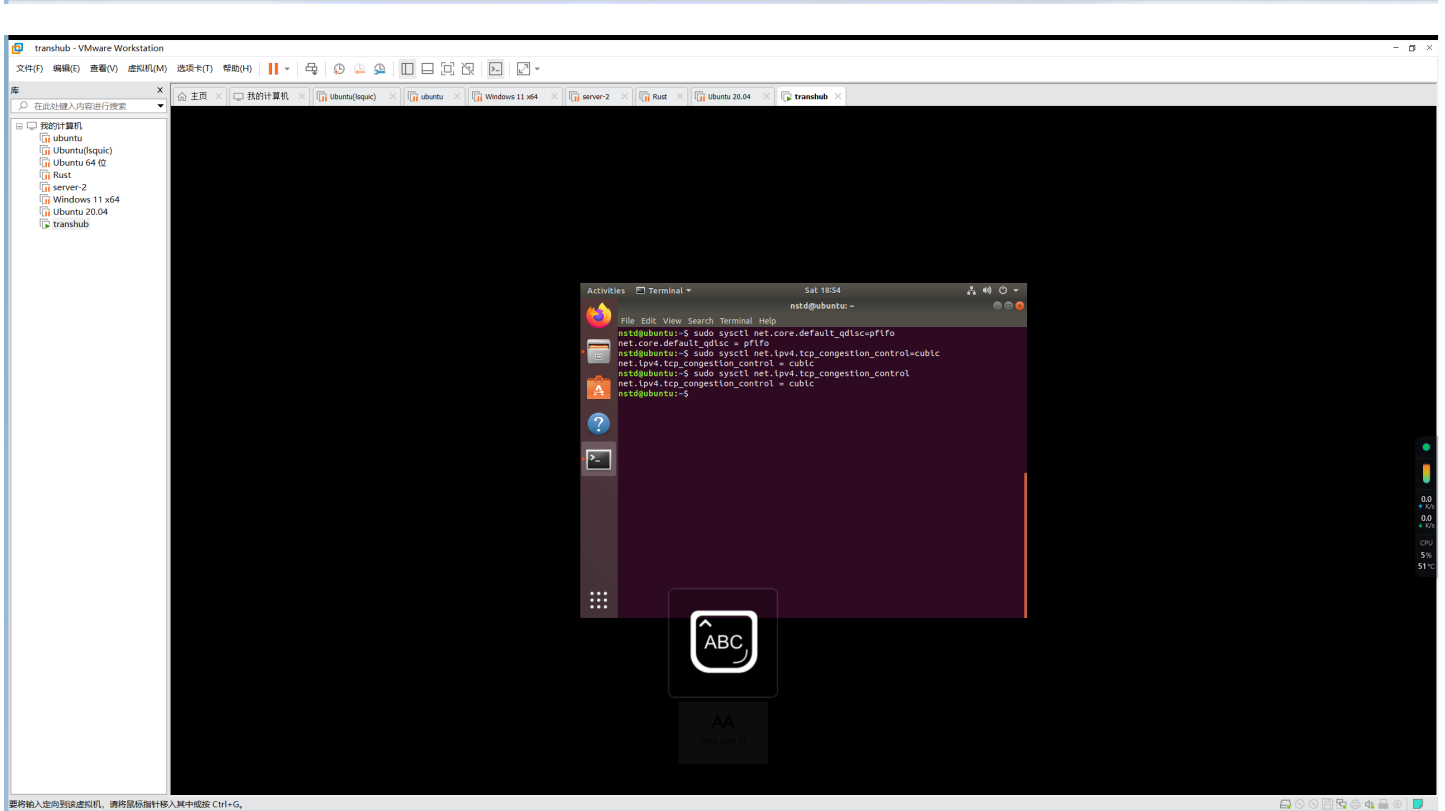
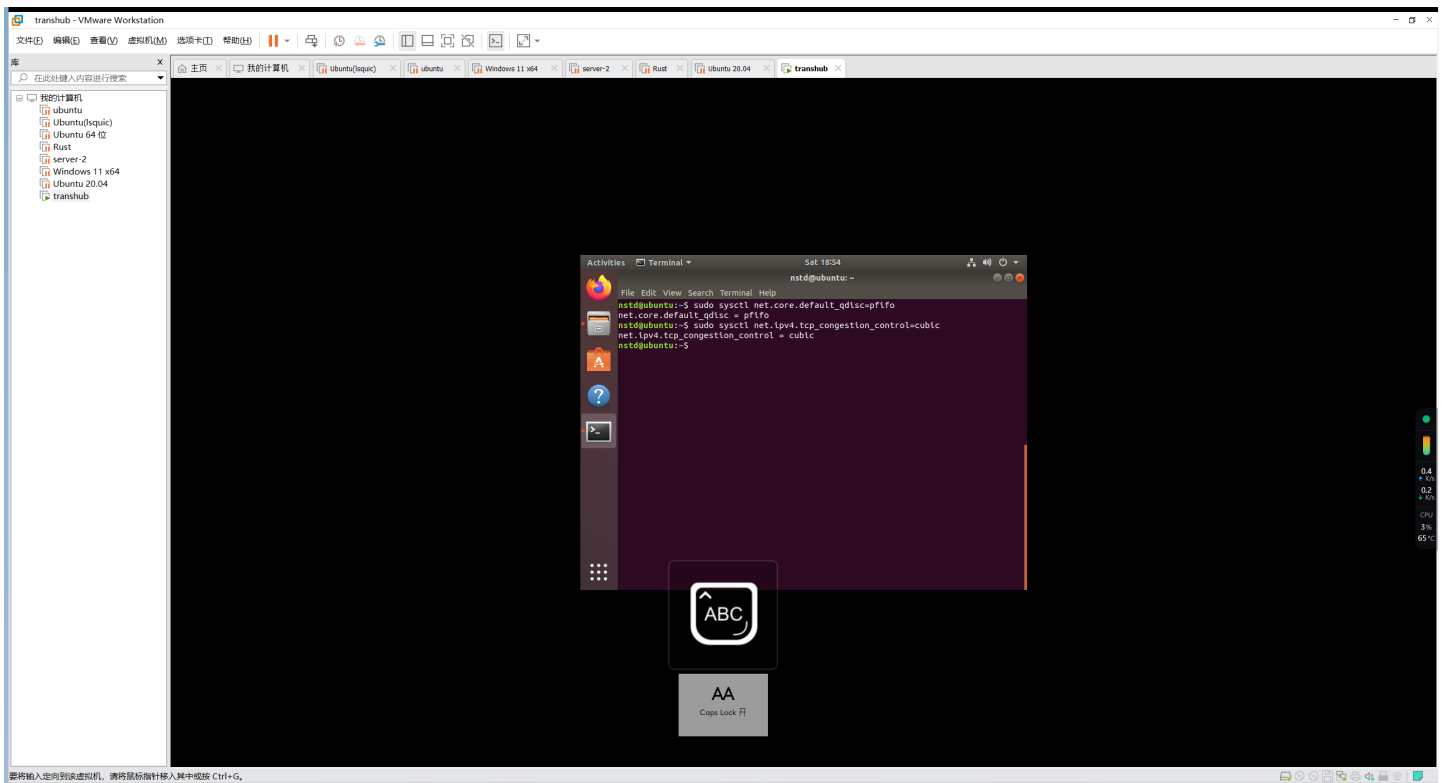
```
1  # 查看系统支持的拥塞控制算法
2  sudo sysctl net.ipv4.tcp_available_congestion_control
3  # 输出结果：表明当前系统支持reno cubic算法
4  # net.ipv4.tcp_available_congestion_control = reno cubic
5
6  # 查看系统当前使用的拥塞控制算法
7  sudo sysctl net.ipv4.tcp_congestion_control
8  # 输出结果：表明当前系统使用cubic算法
9  # net.ipv4.tcp_congestion_control = cubic
10
11 # 切换到BBR
12 sudo sysctl net.core.default_qdisc=fq
13 sudo sysctl net.ipv4.tcp_congestion_control=bbr
14 # 查看是否切换成功
15 sudo sysctl net.ipv4.tcp_congestion_control
16
17 # 切换回CUBIC
18 sudo sysctl net.core.default_qdisc=pfifo
19 sudo sysctl net.ipv4.tcp_congestion_control=cubic
20 # 查看是否切换成功
21 sudo sysctl net.ipv4.tcp_congestion_control
```



要将输入定向到该虚拟机，请将鼠标指针移入其中或按 Ctrl+G。



要将输入定向到该虚拟机，请将鼠标指针移入其中或按 Ctrl+G。



3.2 通过方式二安装的操作过程

💡 在Docker容器中启用BBR算法

附相关知识点：**Docker容器共享主机内核：**

- Docker容器共享宿主机的内核，因此容器内的网络配置（如拥塞控制算法）受宿主机内核限制。
- 如果宿主机内核不支持BBR，容器内也无法使用BBR。

Docker在macOS上和Windows上跑Linux docker都是先套了个虚拟机，虚拟机里跑Linux提供kernel，再在里面跑docker。目前虚拟机在Windows上是Hyper-V或者WSL，macOS上有两套方案。

拥塞控制算法属于内核算法，所以在Docker容器能够使用的拥塞控制算法受宿主机内核确定。bbr算法是较新的算法，并非在所有内核都支持。

使用命令 `sysctl net.ipv4.tcp_available_congestion_control` 查看当前支持的拥塞控制算法：

代码块

```
1  # 查看系统支持的拥塞控制算法
2  sysctl net.ipv4.tcp_available_congestion_control
3  # 输出结果：表明当前系统支持reno cubic算法
4  # net.ipv4.tcp_available_congestion_control = reno cubic
```

- 如是Linux宿主机，可按照3.1节方法开启宿主机bbr算法，在容器中也将变更为bbr算法。
- Mac若是使用Orbstack的容器，可尝试使用reno或cubic算法。目前暂时没找到打开bbr的方法，因为容器内开启bbr算法需要宿主机内核支持。而在Orbstack中，该内核实际上是由Orbstack提供的定制化的内置Linux内核，用户无法修改。
- Windows Docker若是使用WSL后端，理论上可尝试在WSL中修改。本教程未在Windows Docker环境下进行测试，欢迎感兴趣的同学尝试后帮助补充此部分内容。

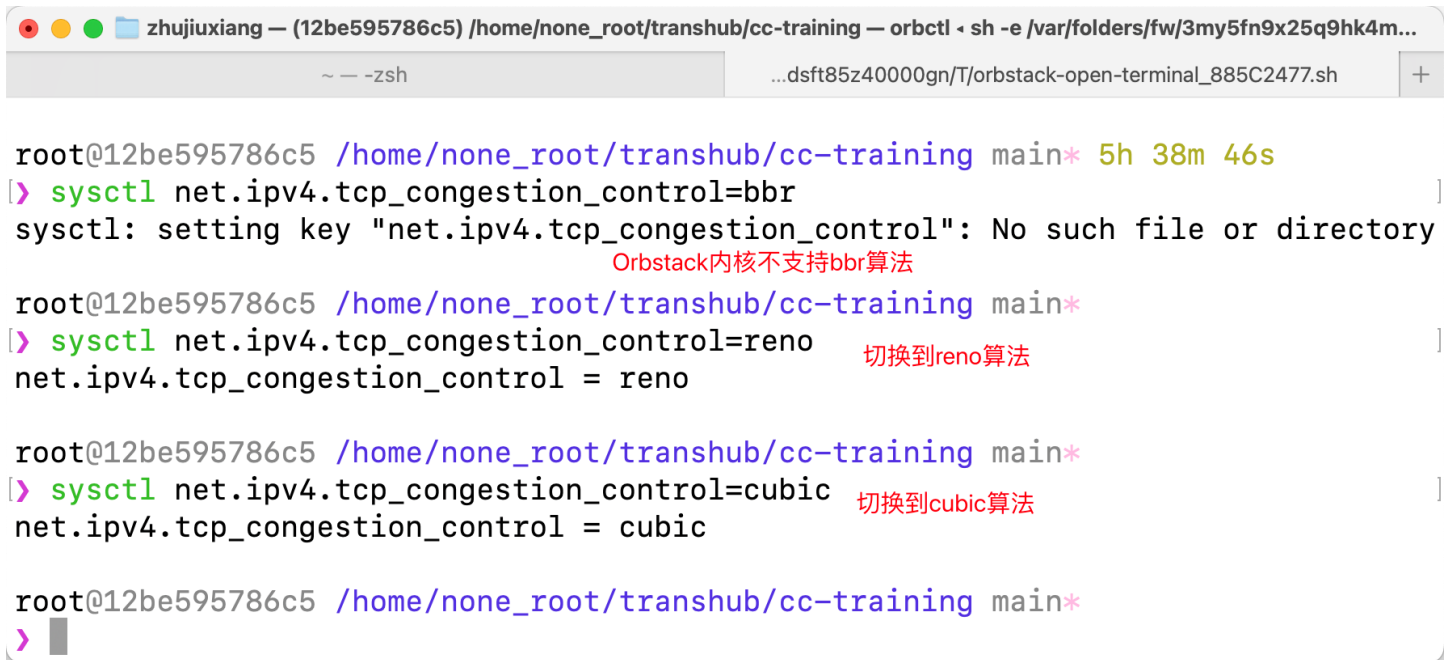
在Docker中切换拥塞控制算法的命令如下：

代码块

```
1  # 查看系统支持的拥塞控制算法
2  sysctl net.ipv4.tcp_available_congestion_control
3  # 输出结果：表明当前系统支持reno cubic算法
4  # net.ipv4.tcp_available_congestion_control = reno cubic
5
6  # 查看系统当前使用的拥塞控制算法
7  sysctl net.ipv4.tcp_congestion_control
8  # 输出结果：表明当前系统使用cubic算法
9  # net.ipv4.tcp_congestion_control = cubic
10
11 #=====若内核支持BBR，使用以下命令切换到BBR以及切回CUBIC
12 # 切换到BBR
13 sysctl net.core.default_qdisc=fq
14 sysctl net.ipv4.tcp_congestion_control=bbr
15 # 查看是否切换成功
16 sysctl net.ipv4.tcp_congestion_control
17
18 # 切回CUBIC
```



```
19 sudo sysctl net.core.default_qdisc=pfifo
20 sudo sysctl net.ipv4.tcp_congestion_control=cubic
21 # 查看是否切换成功
22 sudo sysctl net.ipv4.tcp_congestion_control
23
24 #=====若内核不支持BBR，例如使用Orbstack，使用以下命令切换到RENO以及切回CUBIC
25 # 切换到RENO
26 sysctl net.ipv4.tcp_congestion_control=reno
27
28 # 切换到CUBIC
29 sysctl net.ipv4.tcp_congestion_control=cubic
```



```
zhujiuxiang — (12be595786c5) /home/none_root/transhub/cc-training — orbctl • sh -e /var/folders/fw/3my5fn9x25q9hk4m...
~ — -zsh ...dsft85z40000gn/T/orbstack-open-terminal_885C2477.sh +

root@12be595786c5 /home/none_root/transhub/cc-training main* 5h 38m 46s
[> sysctl net.ipv4.tcp_congestion_control=bbr
sysctl: setting key "net.ipv4.tcp_congestion_control": No such file or directory
Orbstack内核不支持bbr算法

root@12be595786c5 /home/none_root/transhub/cc-training main*
[> sysctl net.ipv4.tcp_congestion_control=reno 切换到reno算法
net.ipv4.tcp_congestion_control = reno

root@12be595786c5 /home/none_root/transhub/cc-training main*
[> sysctl net.ipv4.tcp_congestion_control=cubic 切换到cubic算法
net.ipv4.tcp_congestion_control = cubic

root@12be595786c5 /home/none_root/transhub/cc-training main*
[> █
```

本文档更新时间：2026年03月01日 星期日 15:00